

가상현실

L12. Gaussian Splatting

College of Software

Dong-A University

Prof. Jeongil Seo



3D Gaussian Splatting

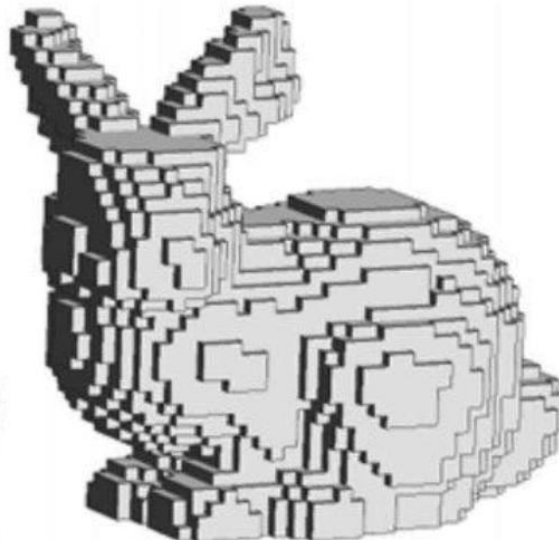
3차원 장면을 수많은 Gaussian(가우시안)으로 표현하고 이를 실시간으로 렌더링하여 고품질의 새로운 시점 영상을 생성하는 3D 장면 표현 기술

Previous Method

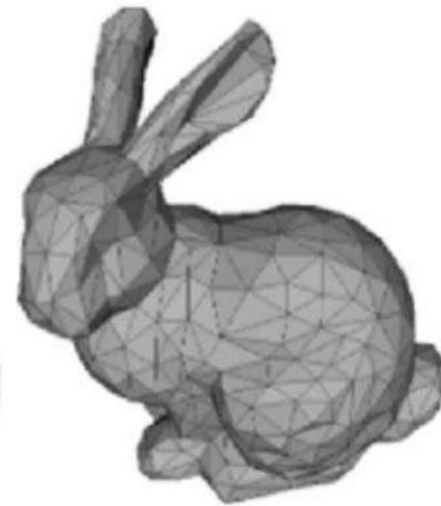
- 종래의 접근 방법들은 Point cloud, Voxel, Mesh 등으로 표현되는 3D 객체를 직접 렌더링함
→ 저장공간이 매우 커지거나, 디테일한 Real-world 표현 어려움



<Point Cloud>



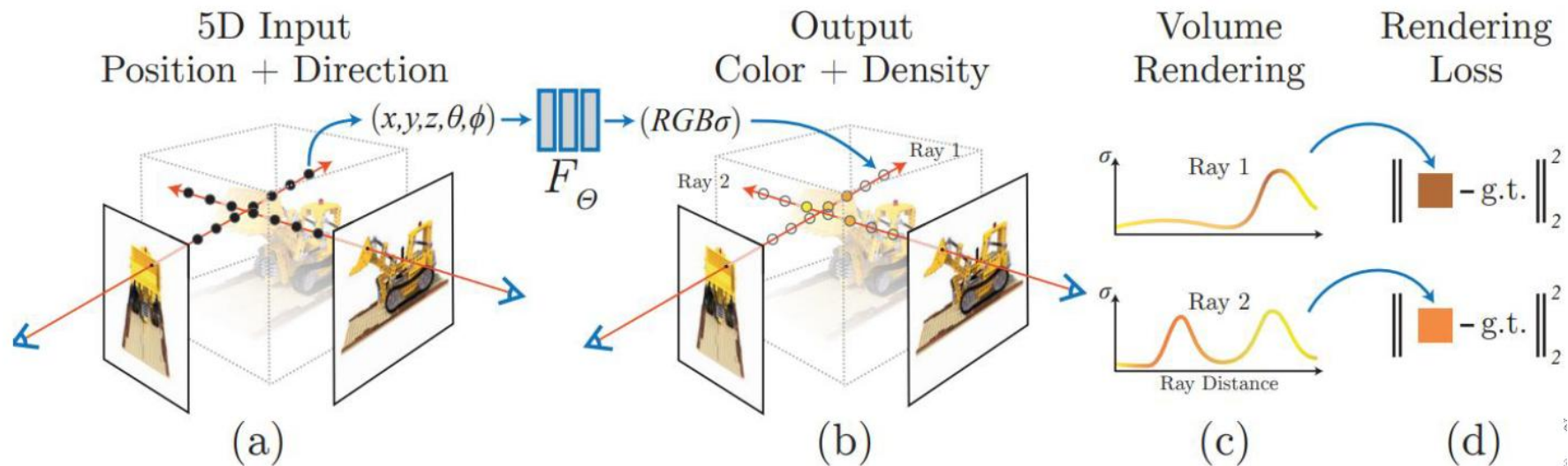
<Voxel>



<Mesh>

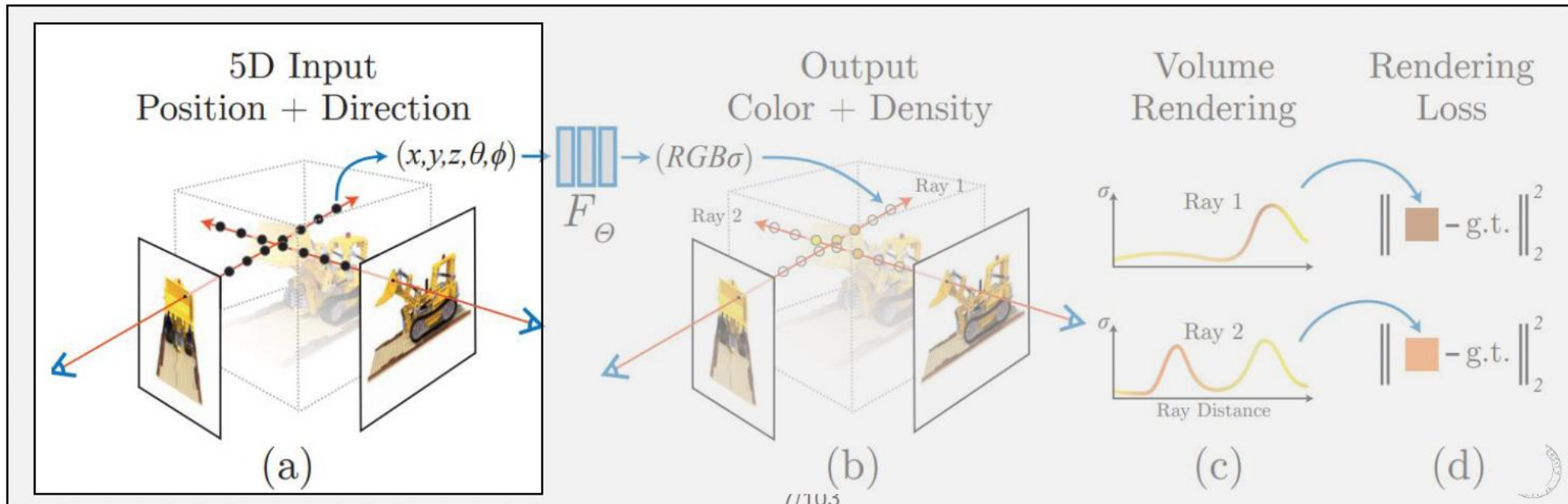
Previous Method - NeRF

- NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis, ECCV 2020
- 입력은 물체의 위치 정보 (x, y, z) 와 방향 (θ, ϕ) 가 존재하는 5차원 데이터임
 - θ : 방위각 (좌우), ϕ : 고도각 (상하)
- 출력은 샘플링된 점들에 대한 RGB 값과 물체의 밀도 σ (Opacity)임
 - MLP를 통해 출력값을 획득함
- 출력값을 volume rendering을 통해 픽셀로 변환



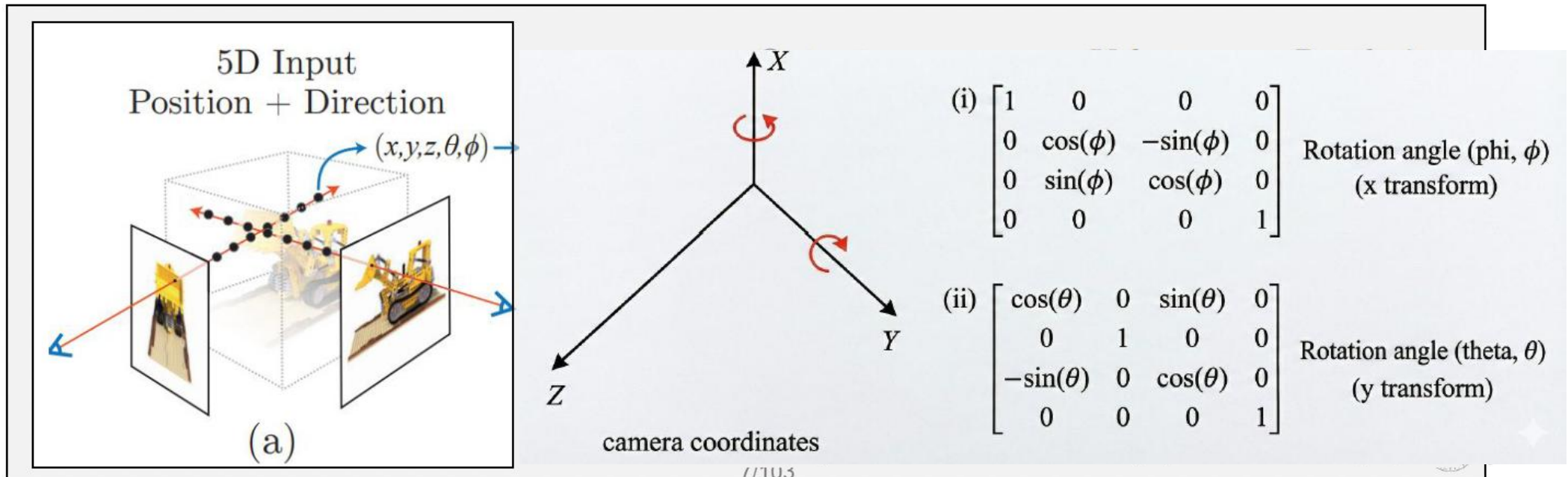
Previous Method - NeRF

- NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis, ECCV 2020
- 입력은 물체의 위치 정보 (x, y, z) 와 카메라 좌표계 parameter (θ, ϕ) 가 존재하는 5차원 데이터임
 θ : 방위각 (좌우), ϕ : 고도각 (상하)

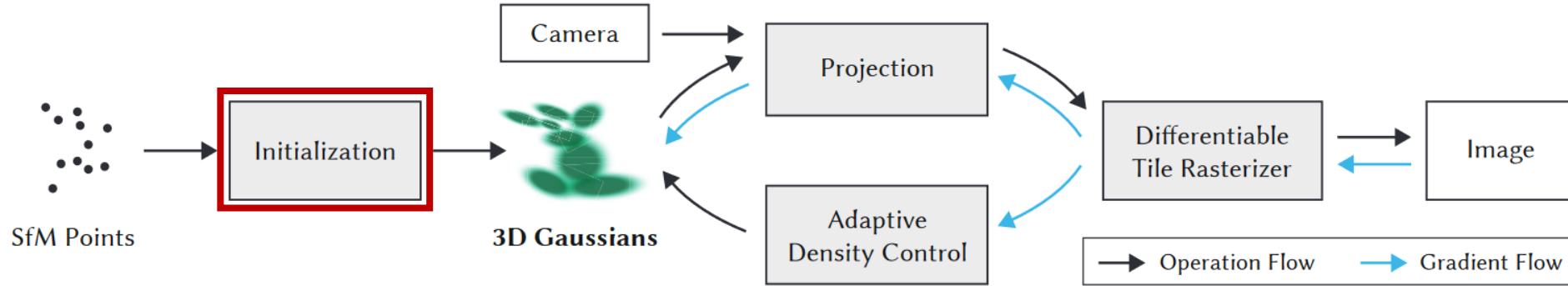


Previous Method - NeRF

- NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis, ECCV 2020
- 입력은 물체의 위치 정보 (x, y, z) 와 카메라 좌표계 parameter (θ, ϕ) 가 존재하는 5차원 데이터임
 θ : 방위각 (좌우), ϕ : 고도각 (상하)



1. SFM Initialization



$$G_i = \{\mu_i, s_i, q_i, o_i, c_i\}$$

$$\mu_i \in \mathbb{R}^3$$

$$s_i \in \mathbb{R}^3 \quad (\text{scale})$$

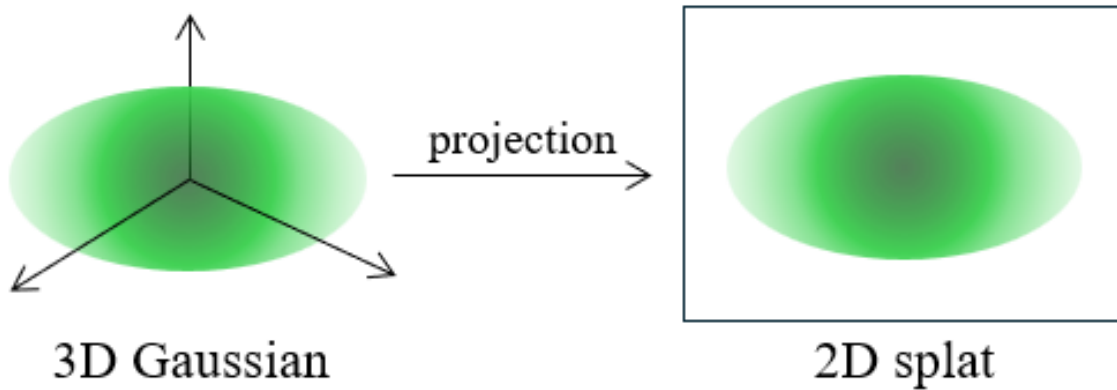
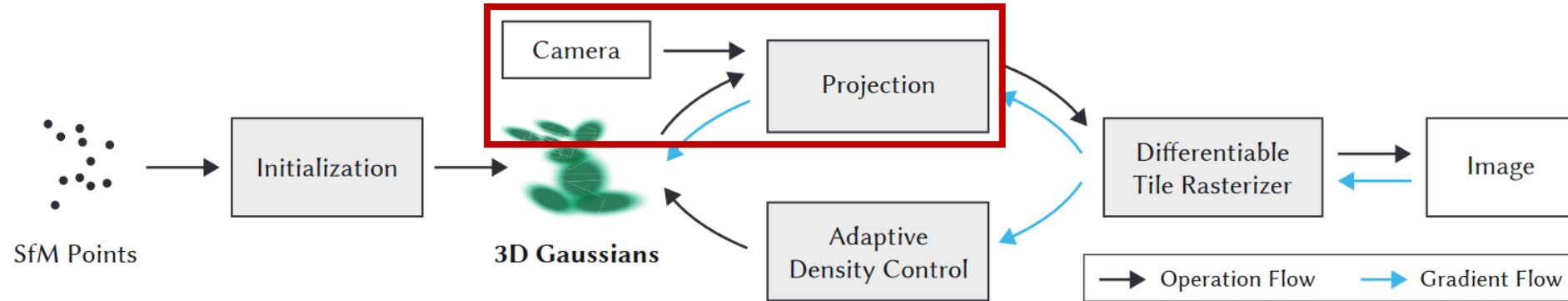
$$q_i \in \mathbb{R}^4 \quad (\text{rotation})$$

$$o_i \in [0,1] \quad (\text{opacity})$$

$$c \quad (\text{color, SH coeff})$$



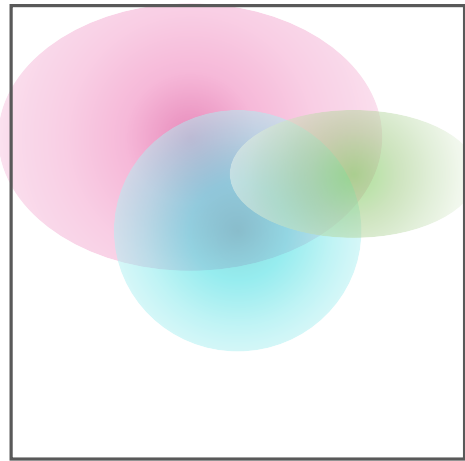
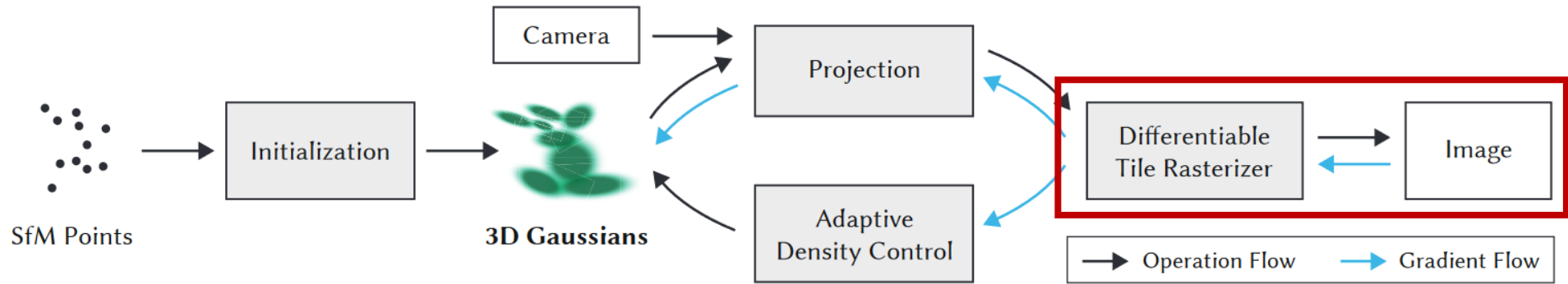
2. 3D Gaussian Projection



$$\Sigma_{3d} = RSS^{\top}R^{\top}$$

$$\Sigma_{2d} = JW\Sigma_{3d}W^{\top}J^{\top}$$

3. Differentiable Rasterization



1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16



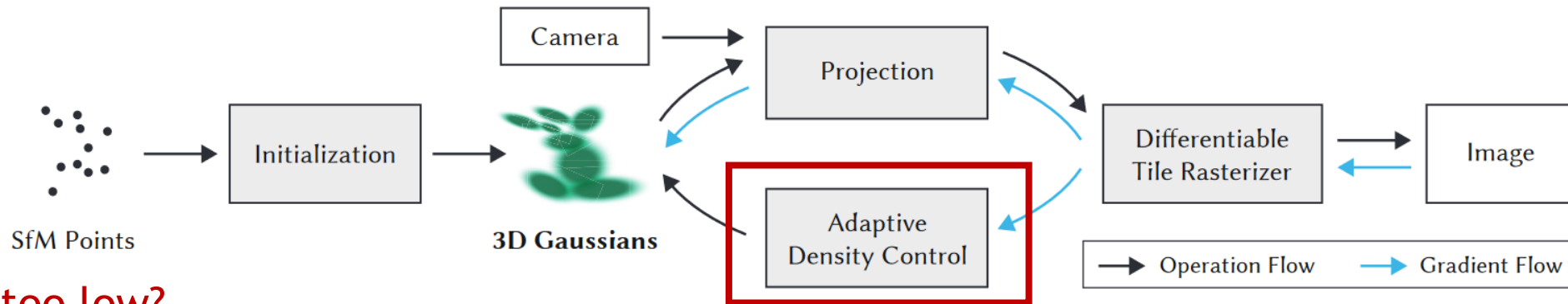
[[ depth, 7 ID], [ depth, 7 ID], [ depth, 7 ID]]
(sorted in depth order)

$$C = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

Indices in a Tile
(sorted in depth order)

$$\alpha_i = o_i e^{-\frac{1}{2}(x - \mu_i)^T \Sigma_{2d} (x - \mu_i)}$$

4. Adaptive Density Control



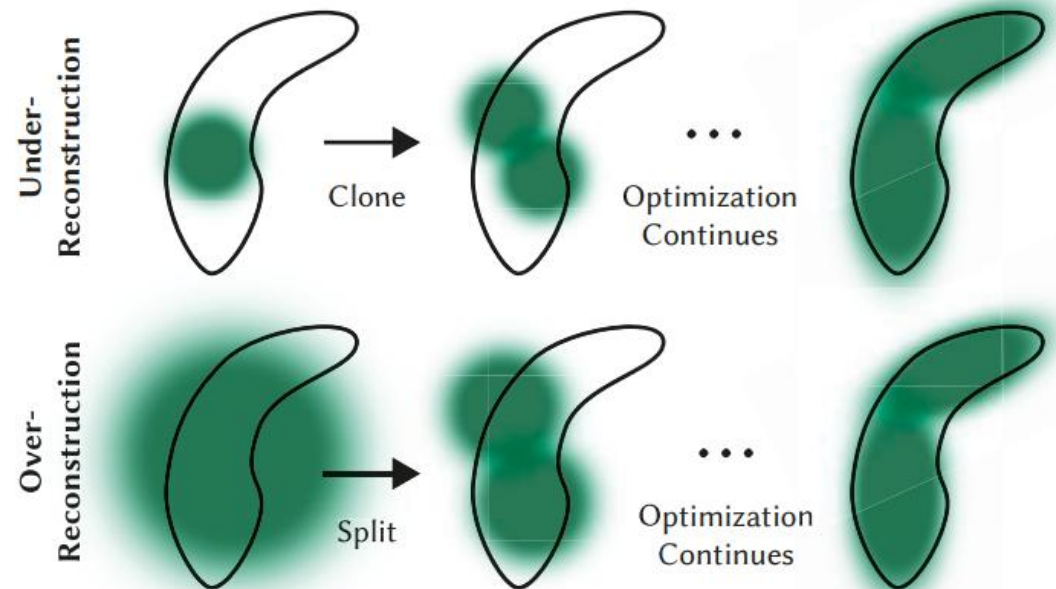
Is opacity too low?

```

if  $\alpha < \epsilon$  or IsTooLarge( $\mu, \Sigma$ ) then           ▷ Pruning
    RemoveGaussian()
end if
if  $\nabla_p L > \tau_p$  then                               ▷ Density Control
    if  $\|S\| > \tau_S$  then                               ▷ Over-reconstruction
        SplitGaussian( $\mu, \Sigma, c, \alpha$ )
    else                                               ▷ Under-reconstruction
        CloneGaussian( $\mu, \Sigma, c, \alpha$ )
    end if
end if
end if
    
```

Is position gradient high?

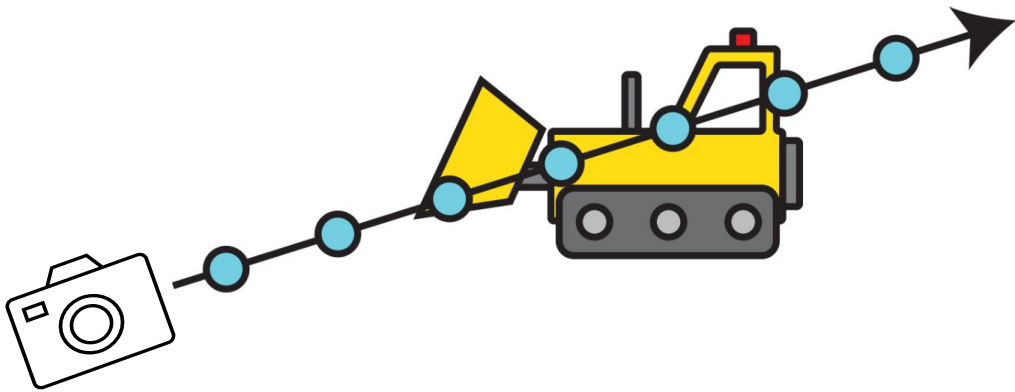
Is Gaussian big?



3DGS Rendering and NeRF Volume Rendering

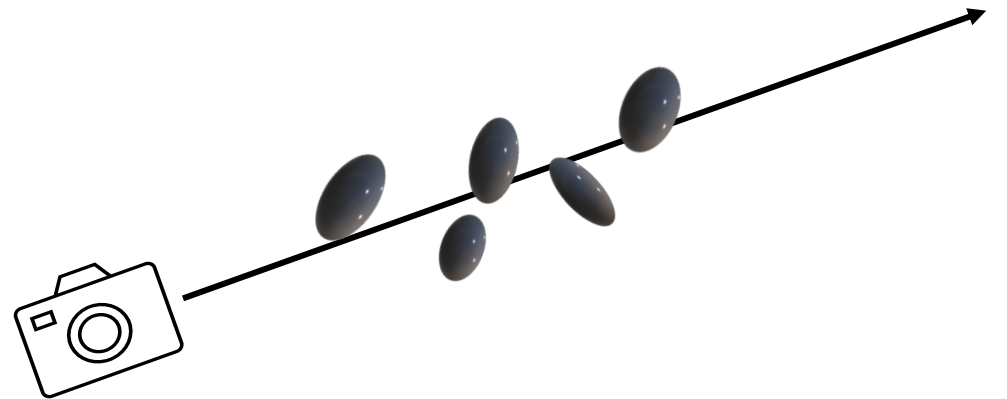
NeRF Volume Rendering

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \delta_i)) c_i$$



3DGS Rendering

$$C = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$





3DGS Performance



1. Super-fast rendering
2. Fast training
3. High quality rendering

3DGS Performance



1. Super-fast rendering
2. Fast training
3. High quality rendering



Too Many Gaussians



Too Many Gaussians -> 1. High Spatial Complexity



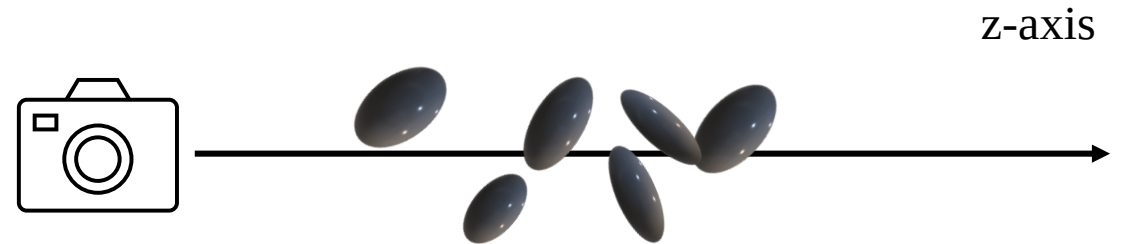
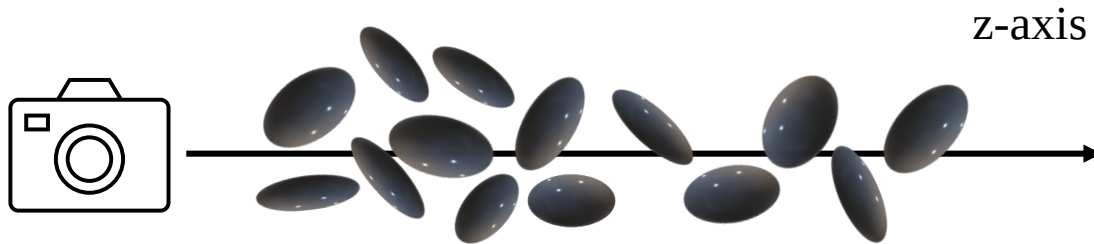
- 59 x 4 bytes for a single Gaussian
- Bicycle scene uses more than 5M Gaussians -> 1.3GB



Too Many Gaussians -> 2. Slow Rendering



$$C = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$



Too Many Gaussians -> 3. Overfitting Issue

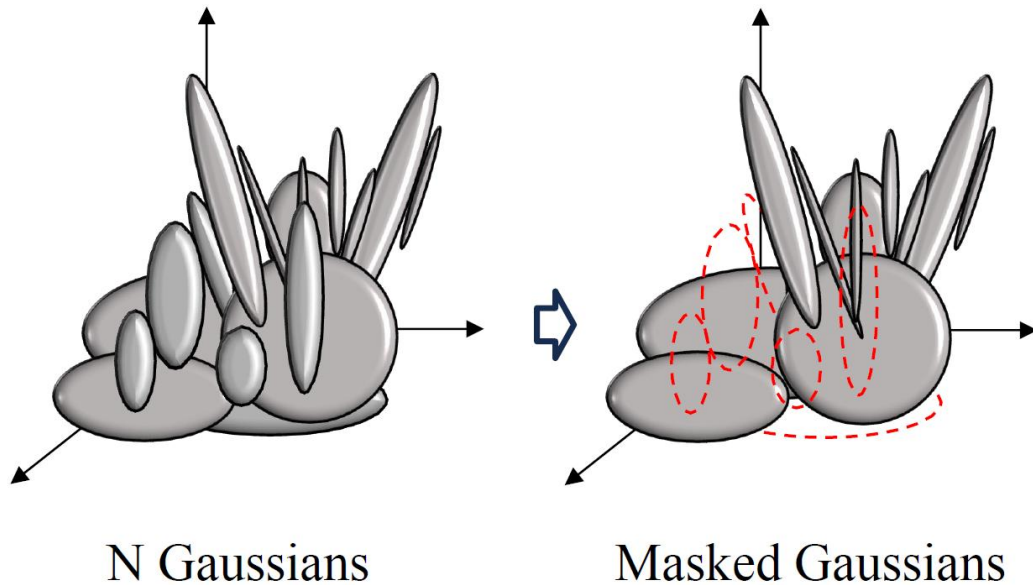


- The more Gaussians -> the more optimized to training views



Gaussian Volume Mask

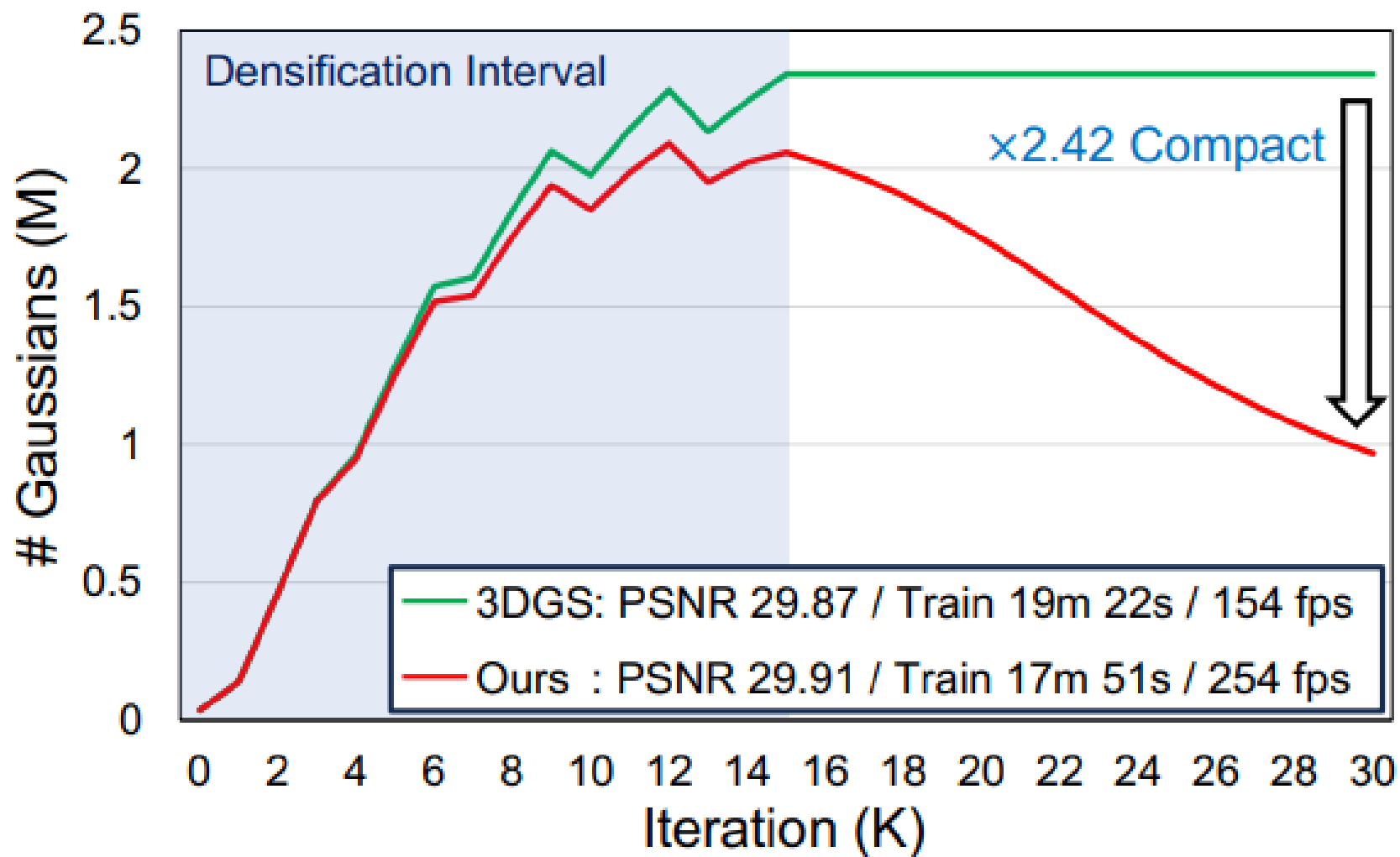
- Learnable binary masks
- Straight-through estimator (STE) for a thresholding function



$$M_n = \text{sg} \left(\begin{matrix} 1[\sigma(m_n) > \epsilon] \\ -\sigma(m_n) \end{matrix} \right) + \sigma(m_n)$$

$$\hat{s}_n = M_n s_n \quad \hat{o}_n = M_n o_n$$

Gaussian Volume Mask



Compressing Gaussian Attributes #1 (Shape)

1. Gaussian shapes are not that diverse....
2. It doesn't seem to show local similarity....



Compressing Gaussian Attributes #1 (Shape)

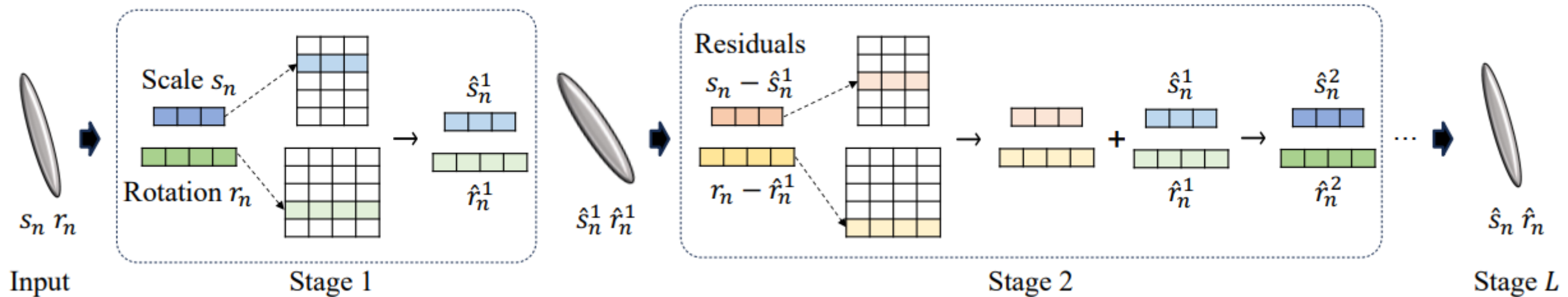
1. Gaussian shapes are not that diverse....
2. It doesn't seem to show local similarity....

Vector Quantization



Compressing Gaussian Attributes #1 (Shape)

- Residual VQ for scaling and rotation

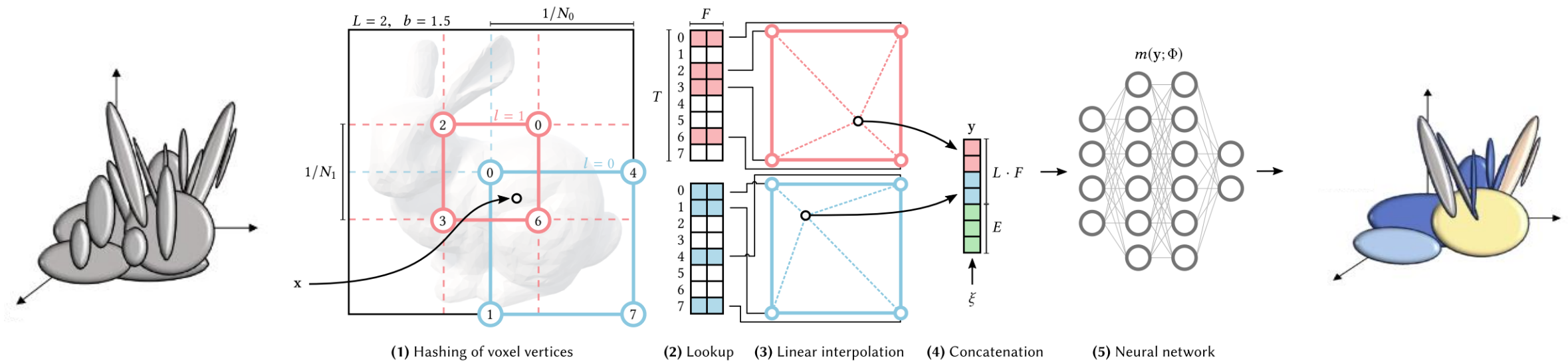


Compressing Gaussian Attributes #2 (Color)

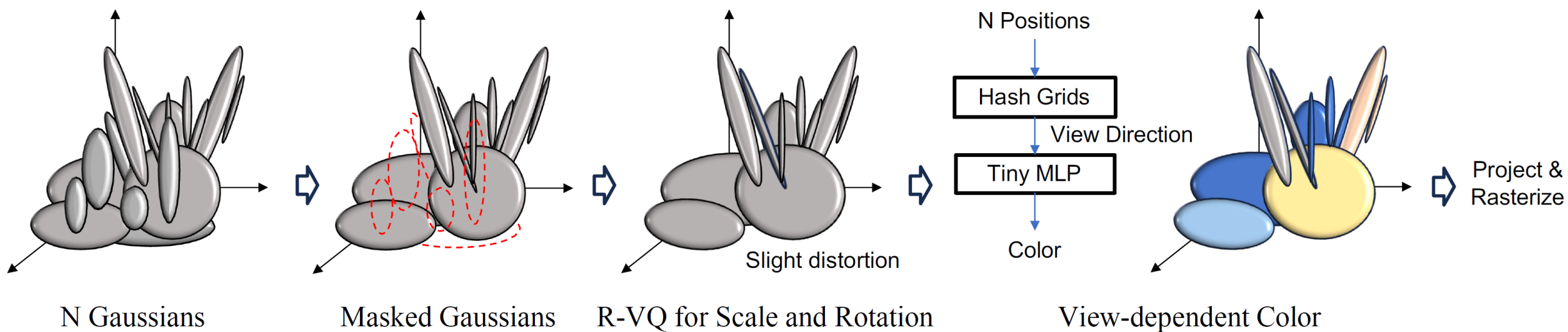
- One Gaussian $\rightarrow$ 48 dim for SH coefficients (3-degrees)
- It usually shows locality: neighboring Gaussians $\rightarrow$ similar colors
- We need a compact representation + fast query.....

Compressing Gaussian Attributes #2 (Color)

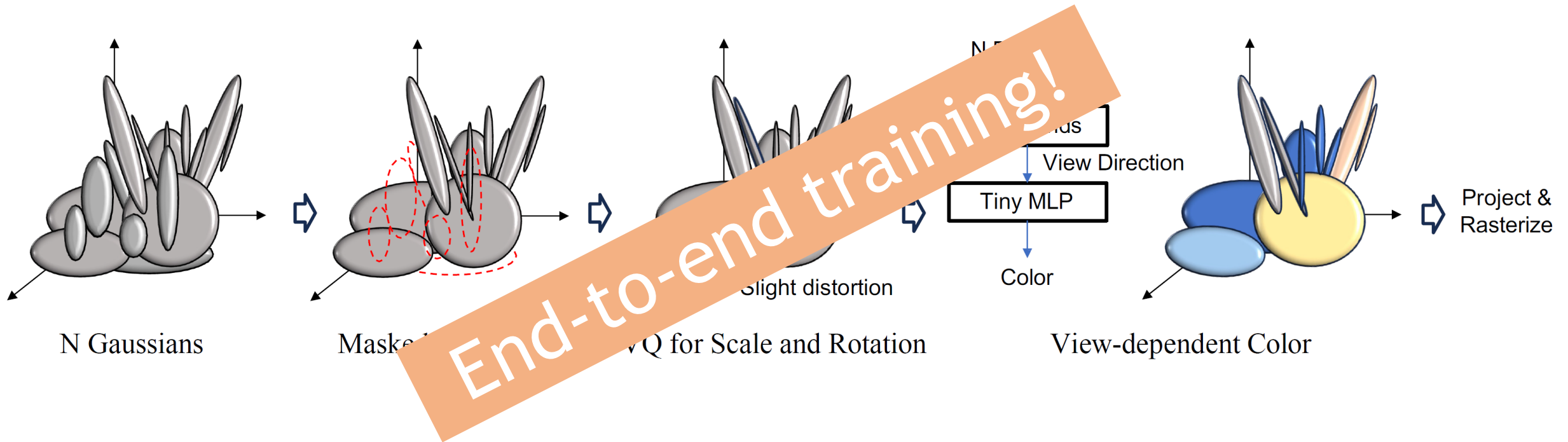
- Instant-NGP: Hash-based Grid + Small MLP
- **Compact and Fast**



Compact 3DGS



Compact 3DGS



Compact 3DGS - Quantitative Results

Dataset	Mip-NeRF 360						Tanks&Temples					
Method	PSNR	SSIM	LPIPS	Train	FPS	Storage	PSNR	SSIM	LPIPS	Train	FPS	Storage
Plenoxels	23.08	0.626	0.463	25m 49s	6.79	2.1 GB	21.08	0.719	0.379	25m 05s	13.0	2.3 GB
INGP-base	25.30	0.671	0.371	05m 37s	11.7	13 MB	21.72	0.723	0.330	05m 26s	17.1	13 MB
INGP-big	25.59	0.699	0.331	07m 30s	9.43	48 MB	21.92	0.745	0.305	06m 59s	14.4	48 MB
Mip-NeRF 360	27.69	0.792	0.237	48h	0.06	8.6 MB	22.22	0.759	0.257	48h	0.14	8.6 MB
3DGS	27.21	0.815	0.214	41m 33s	134	734 MB	23.14	0.841	0.183	26m 54s	154	411 MB
3DGS*	27.46	0.812	0.222	24m 07s	120	746 MB	23.71	0.845	0.178	13m 51s	160	432 MB
Ours	27.08	0.798	0.247	33m 06s	128	48.8 MB	23.32	0.831	0.201	18m 20s	185	39.4 MB
Ours+PP	27.03	0.797	0.247	-	-	26.2 MB	23.32	0.831	0.202	-	-	18.9 MB

Huffman
Coding / bit
quantization



Static Scenes

3DGS 25.10 PSNR / 1.4 GB



3DGS 29.06 PSNR / 774 MB



Ours 24.72 PSNR / 42 MB



Ours 29.16 PSNR / 28 MB



Static Scenes

3DGS 25.10 PSNR / 1.4 GB



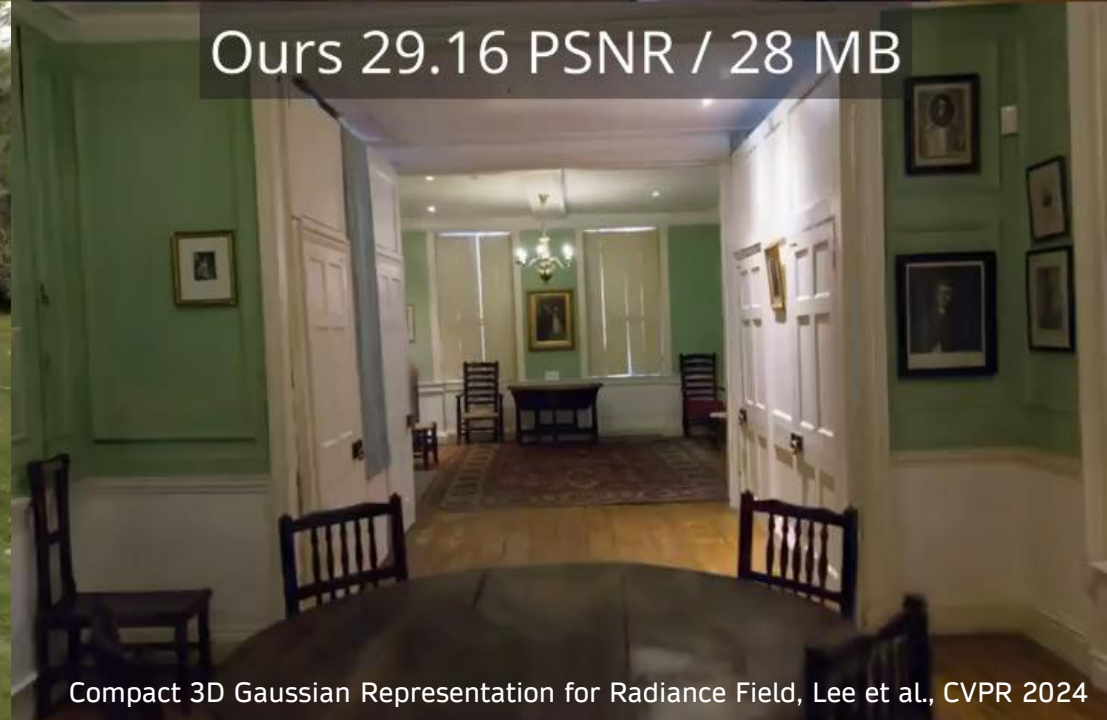
3DGS 29.06 PSNR / 774 MB



Ours 24.72 PSNR / 42 MB



Ours 29.16 PSNR / 28 MB



STG 296 MB

0

Dynamic Scenes



STG 739 MB



Ours 19.6 MB



Ours 111 MB



STG 296 MB

Dynamic Scenes



Ours 19.6 MB



STG 739 MB



Ours 111 MB



Mini-Splatting

- The densification incurs inefficient Gaussian distributions
- Blur split, depth initialization, and simplification
- Point cloud compression (RAHT)



3DGS (6.1mil, 66 FPS)
Train: 35min, PSNR: 25.2dB



Mini-Splatting-D (6.0mil, 75 FPS)
Train: 33min, PSNR: 25.5dB



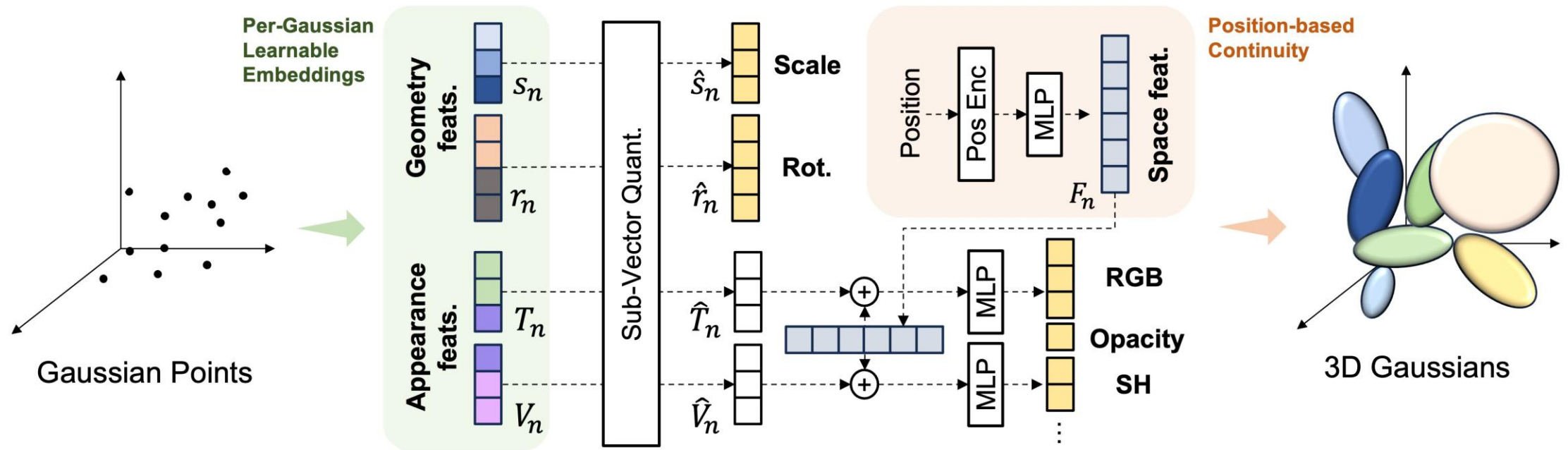
Mini-Splatting (0.6mil, 430 FPS)
Train: 17min, PSNR: 25.2dB

Mini-Splatting

Dataset		Mip-NeRF 360				Tanks&Temples				Deep Blending			
Method	Metric	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	Num	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	Num	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	Num
Plenoxels	[40]	0.626	23.08	0.463	-	0.719	21.08	0.379	-	0.795	23.06	0.510	-
INGP-Big	[26]	0.699	25.59	0.331	-	0.745	21.92	0.305	-	0.817	24.96	0.390	-
mip-NeRF 360	[2]	0.792	27.69	0.237	-	0.759	22.22	0.257	-	0.901	29.40	0.245	-
Zip-NeRF	[3]	0.828	28.54	0.189	-	-	-	-	-	-	-	-	-
3DGS	[17]	0.815	27.21	0.214	3.36	0.841	23.14	0.183	1.78	0.903	29.41	0.243	2.98
3DGS	[17]*	0.815	27.47	0.216	3.35	0.848	23.66	0.176	1.84	0.904	29.54	0.244	2.82
Mini-Splatting-D		0.831	27.51	0.176	4.69	0.853	23.23	0.140	4.28	0.906	29.88	0.211	4.63
Mini-Splatting		0.822	27.34	0.217	0.49	0.835	23.18	0.202	0.20	0.908	29.98	0.253	0.35

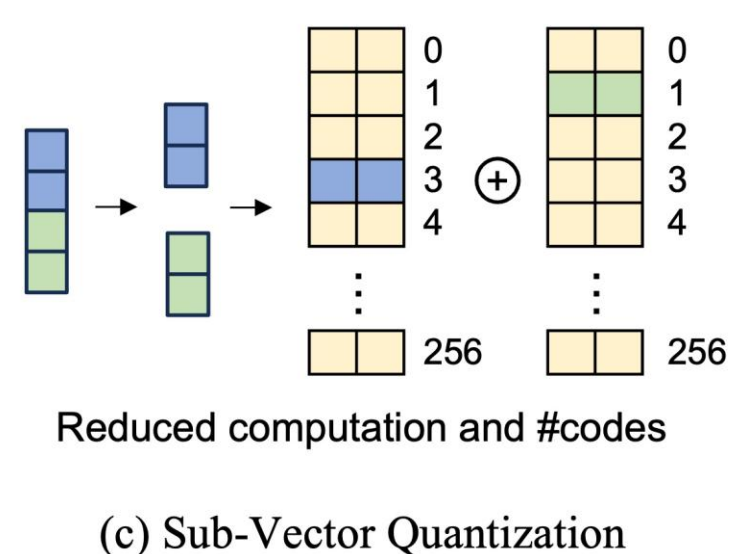
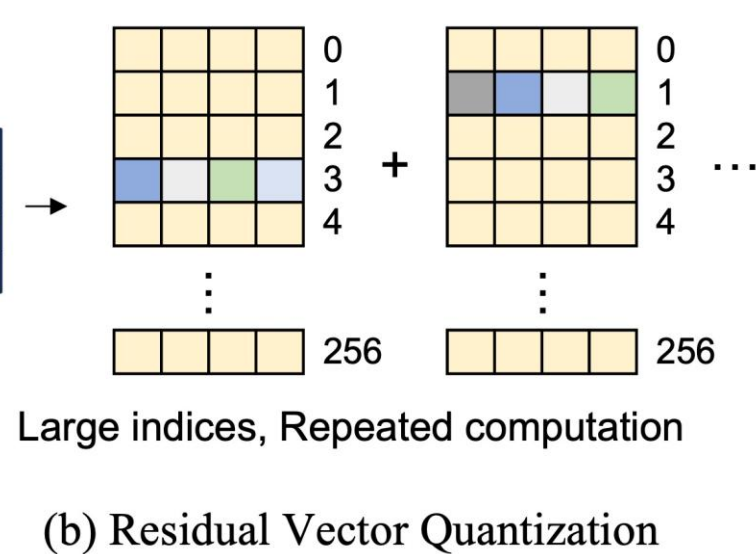
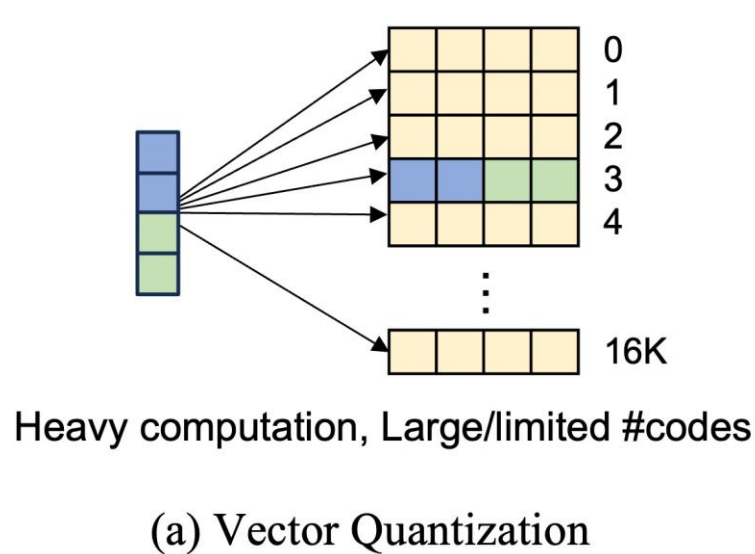
Optimized Minimal 3D Gaussian Splatting

- Minimal number of Gaussians
- Sub-vector Quantization



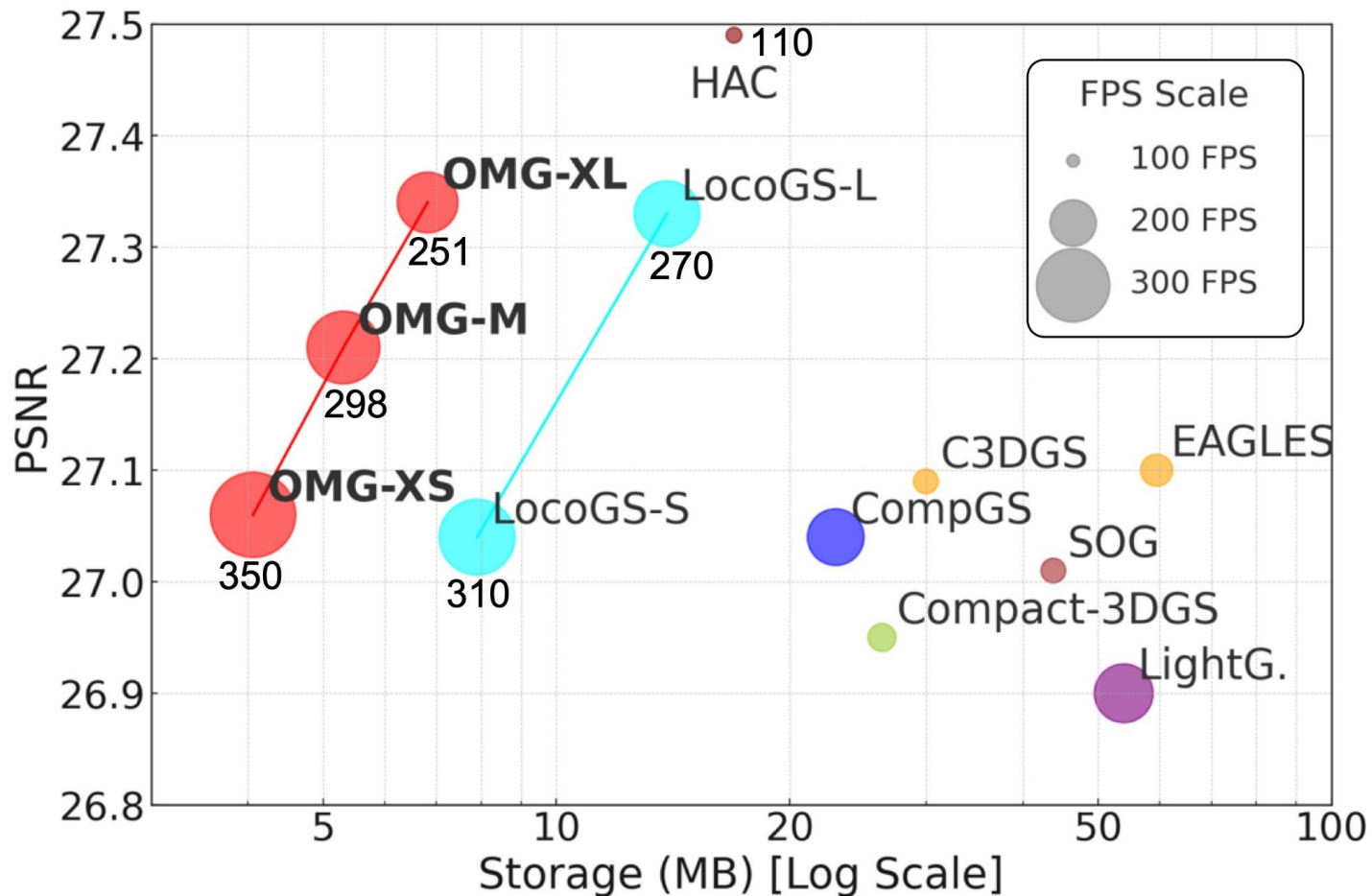
Optimized Minimal 3D Gaussian Splatting

- Minimal number of Gaussians
- Sub-vector Quantization



Optimized Minimal 3D Gaussian Splatting

Rate-Distortion Curve with Rendering Speed



Results

3DGS 26.66PSNR/1.0GB/158FPS



3DGS 31.59PSNR/278MB/230FPS



OMG-XS 27.00PSNR/5.0MB/708FPS



OMG-XS 31.09PSNR/2.7MB/648FPS



Results

3DGS 26.66PSNR/1.0GB/158FPS



3DGS 31.59PSNR/278MB/230FPS



OMG-XS 27.00PSNR/5.0MB/708FPS



OMG-XS 31.09PSNR/2.7MB/648FPS



3D Surface Reconstruction

3D Reconstruction from Multi-view Images

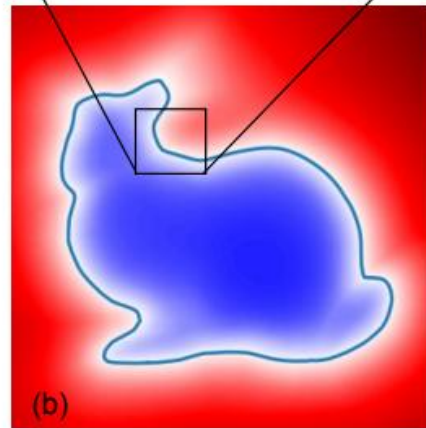
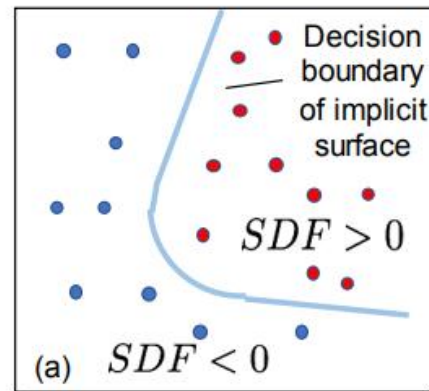
- It is a fundamental problem in computer vision and graphics
- Roughly two categories
 - Point or surface-based reconstruction
 - Volumetric reconstruction



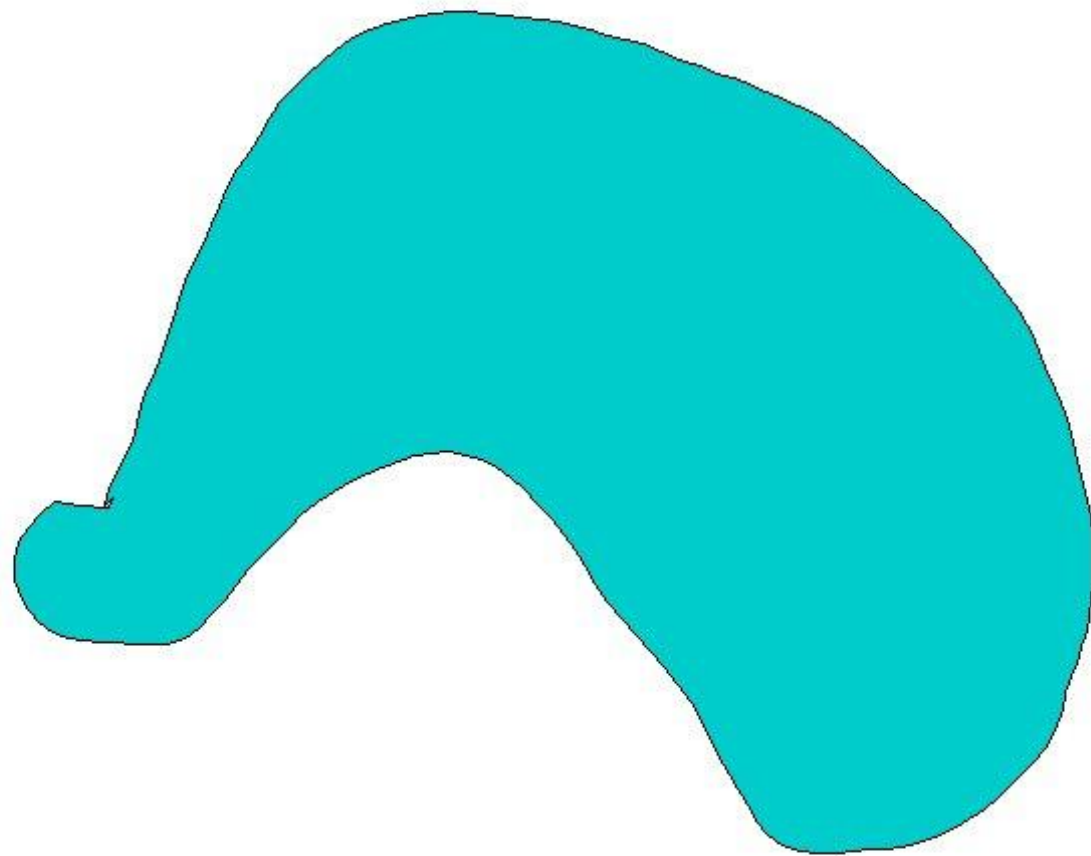
(Image credit: Neuralangelo)

Meshing

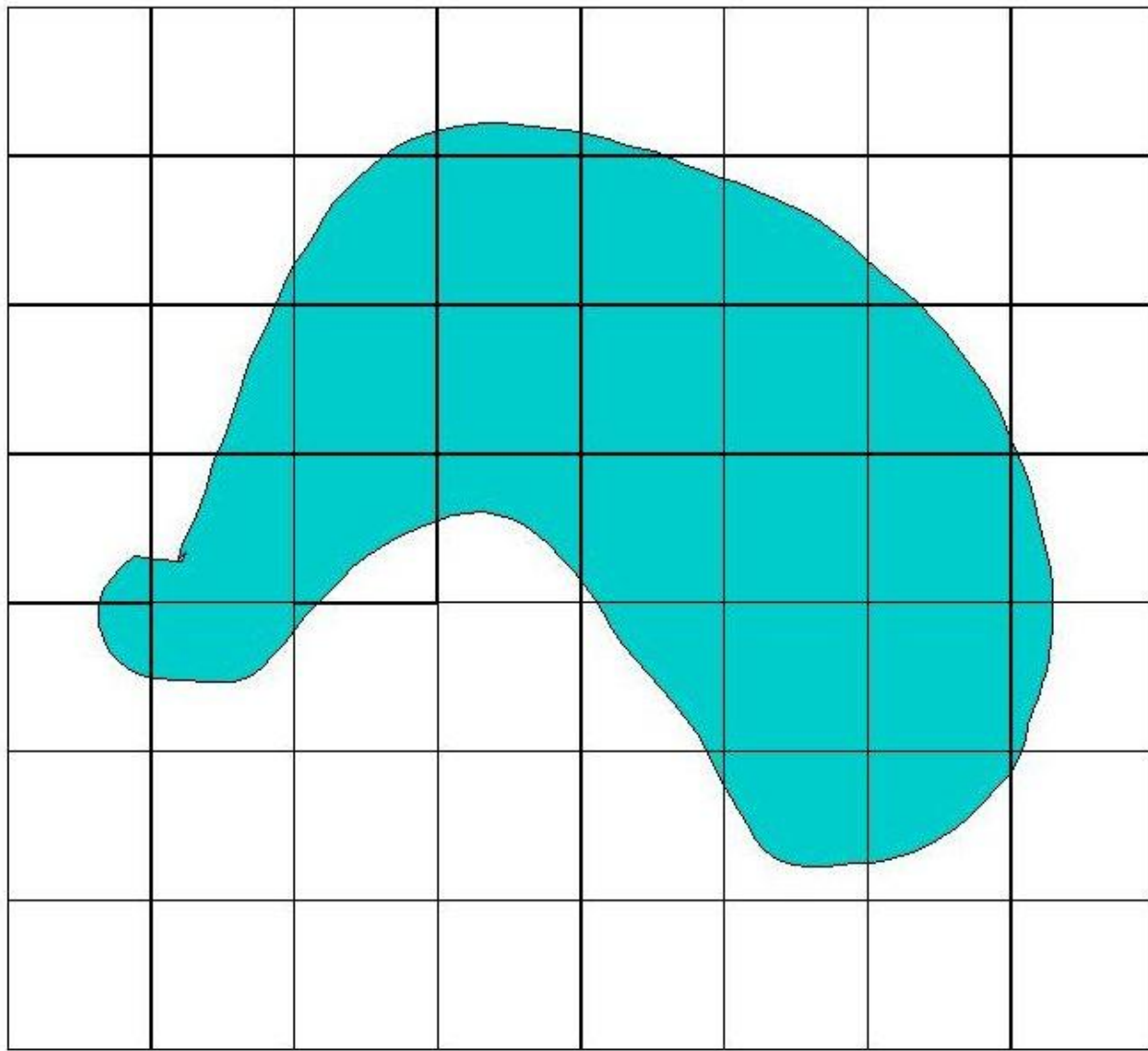
- Signed Distance Fields



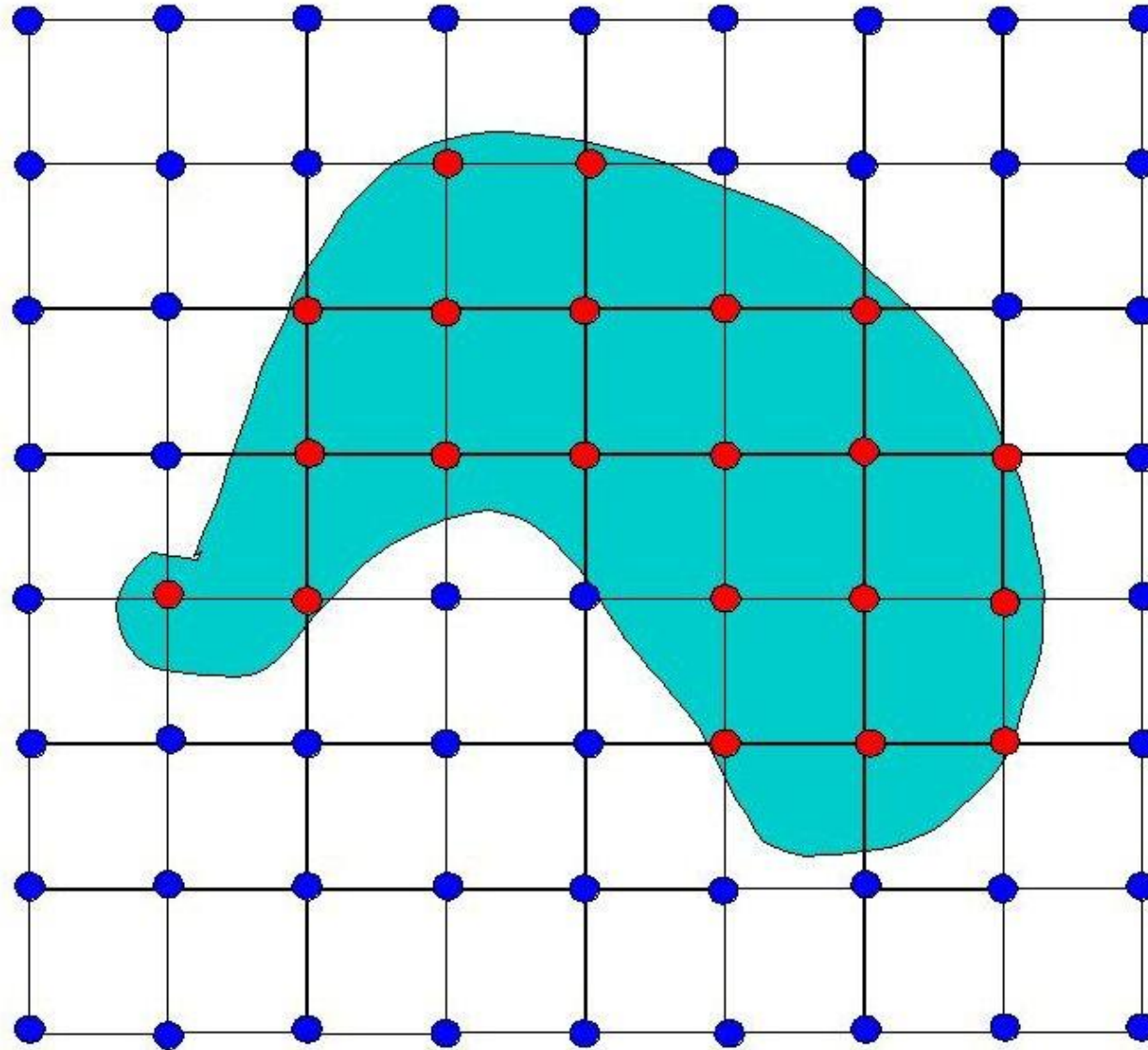
Marching Cube



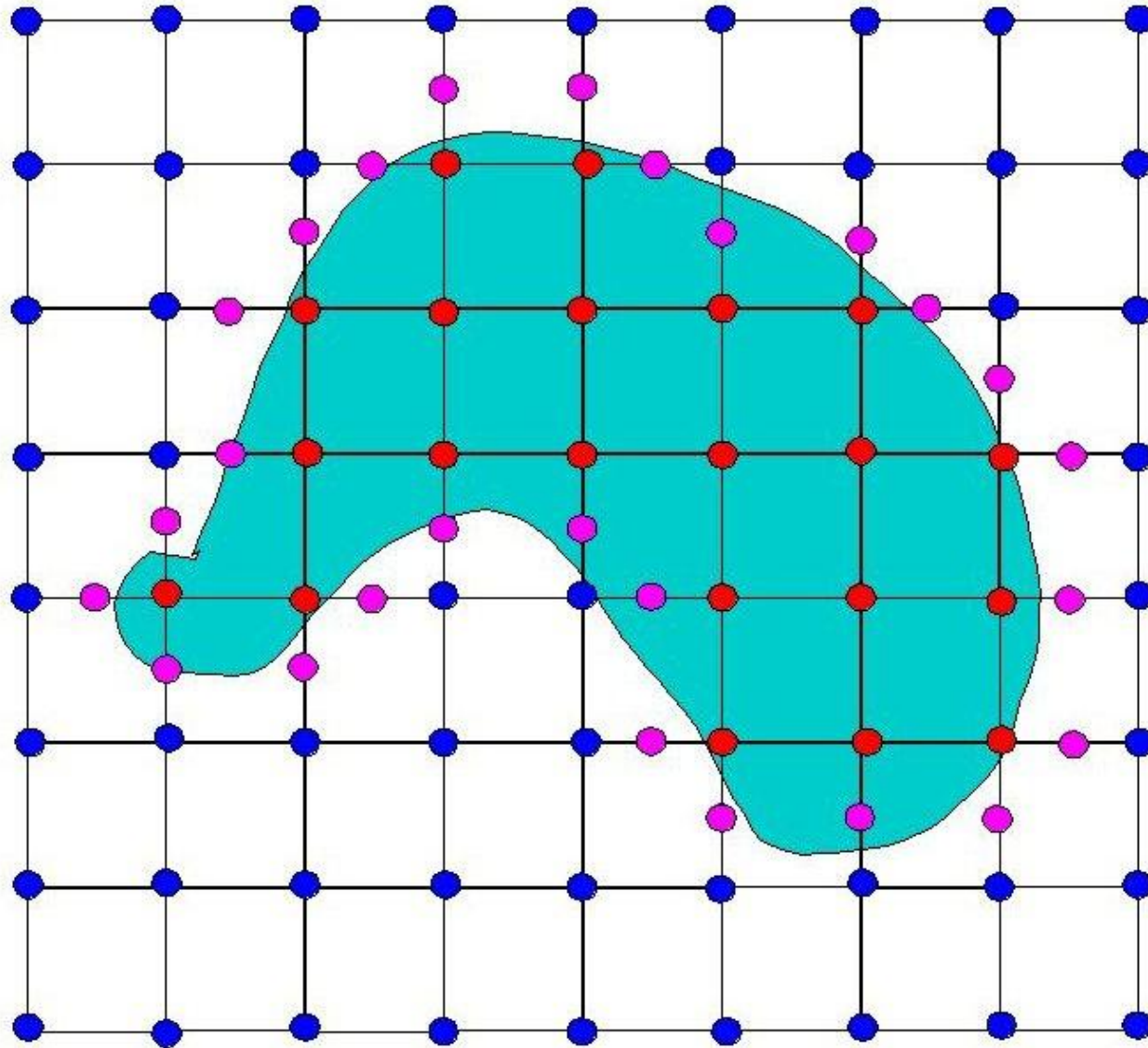
Marching Cube



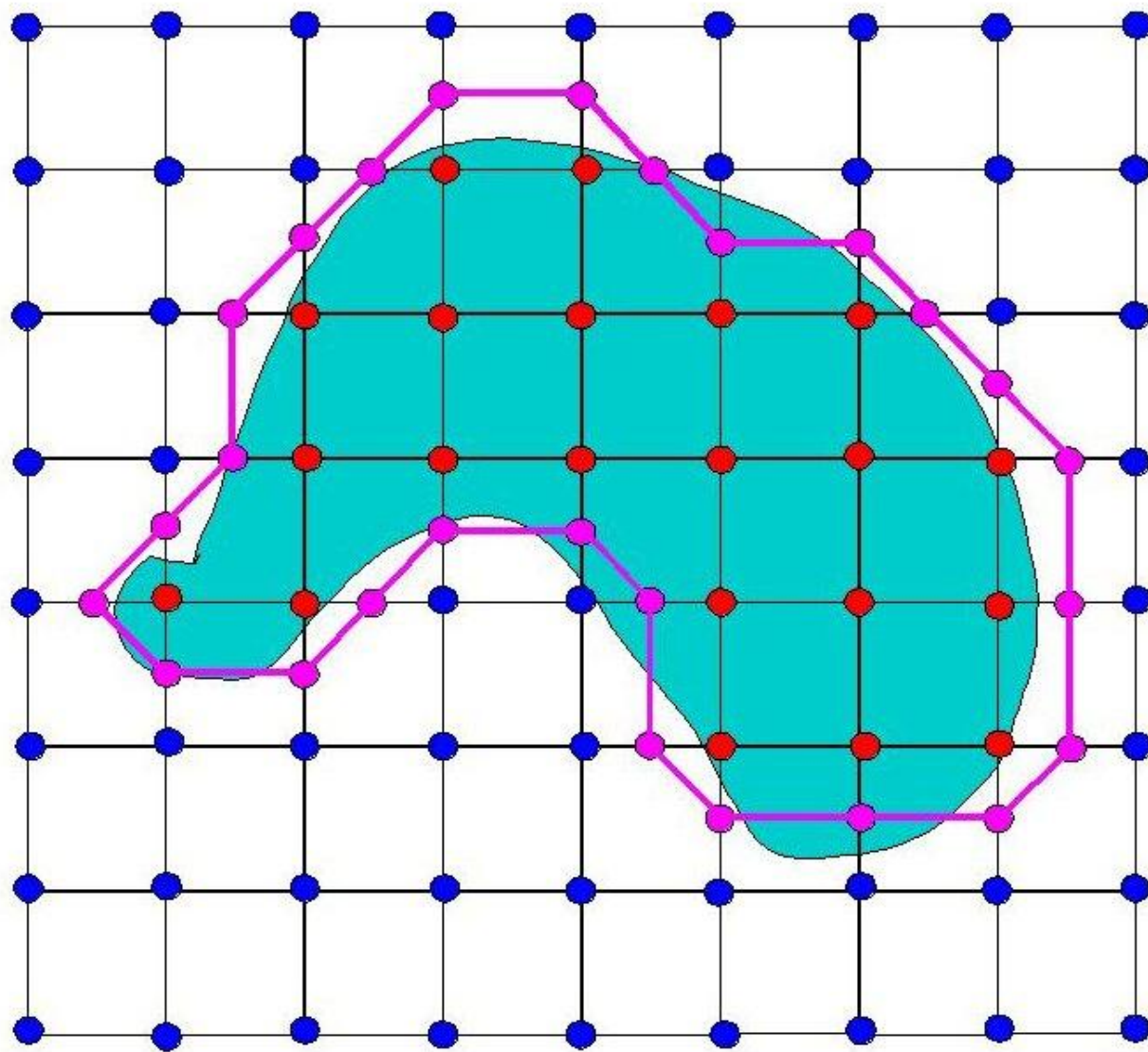
Marching Cube



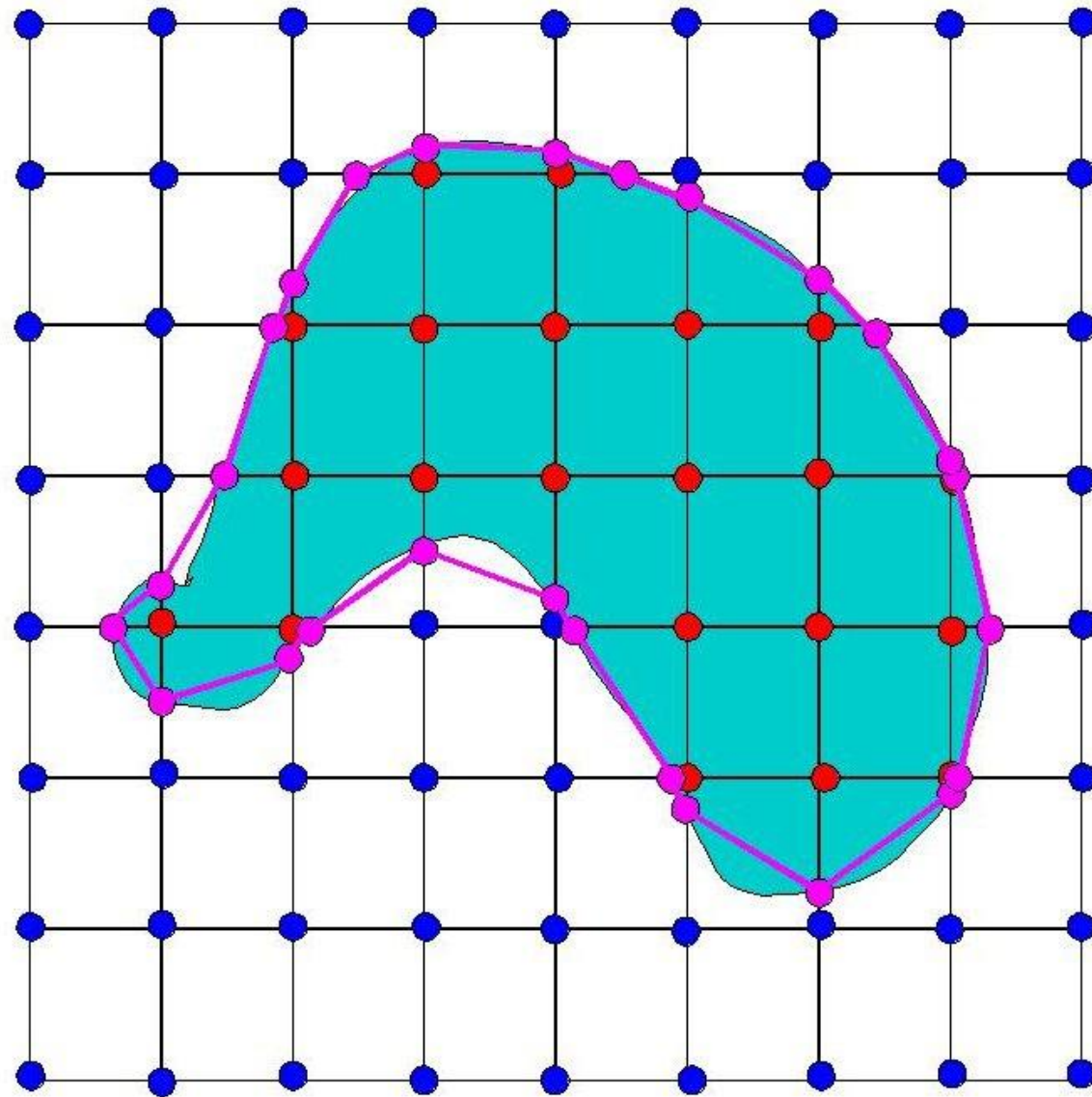
Marching Cube



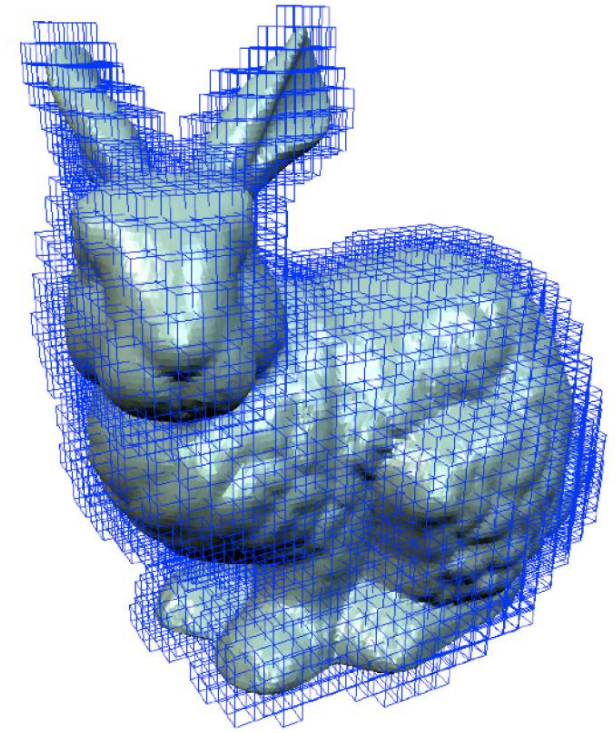
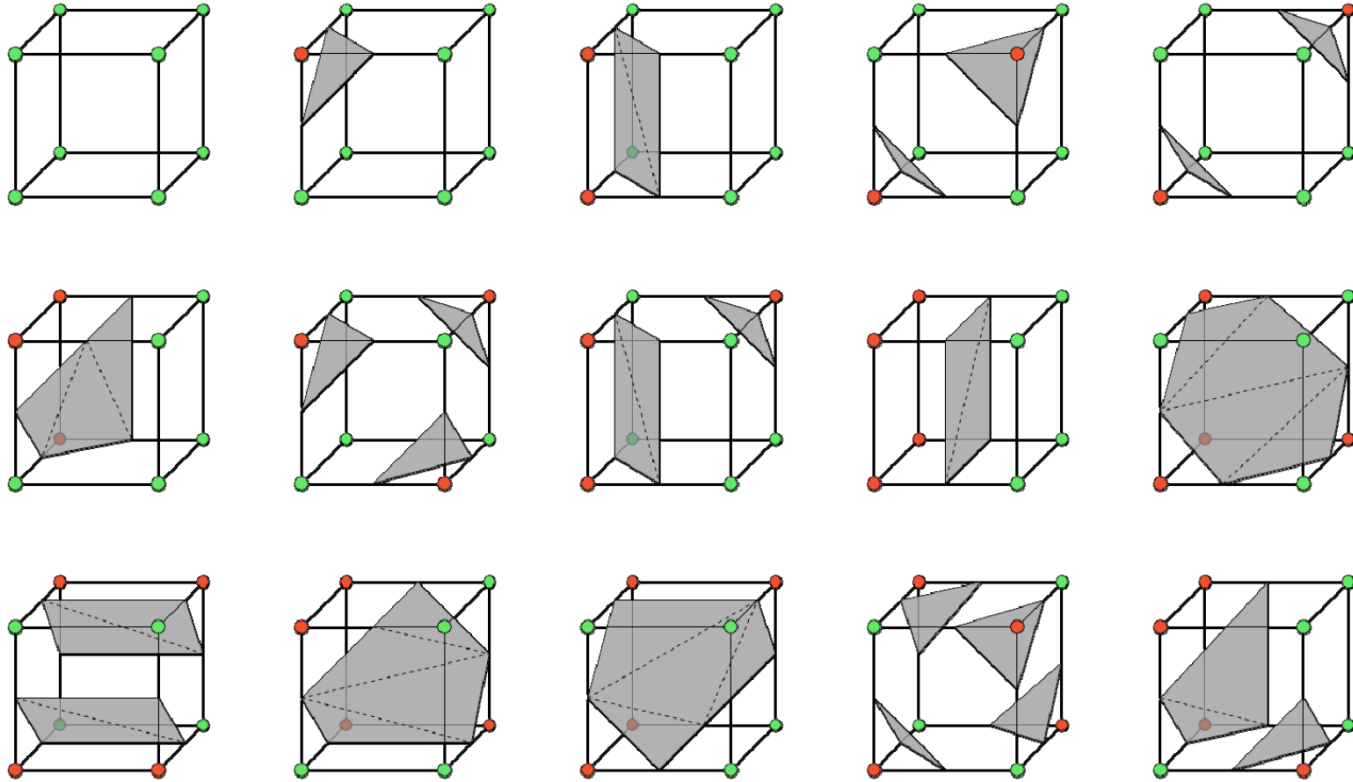
Marching Cube



Marching Cube



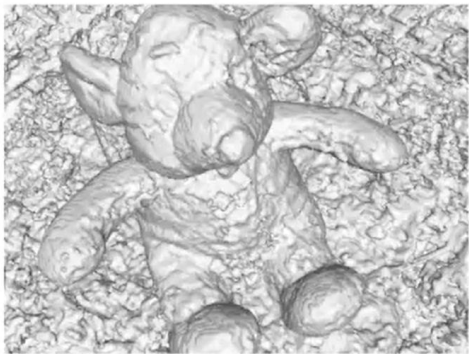
Marching Cube in 3D



Surface Reconstruction using NeRF

- The density-based scene representation lacks sufficient constraints
- The surfaces extracted as level sets of the underlying volume density are usually non-smooth and contain artifacts

$$(R, G, B, \sigma) = f(x, y, z, \theta, \phi)$$



$\sigma = 1$



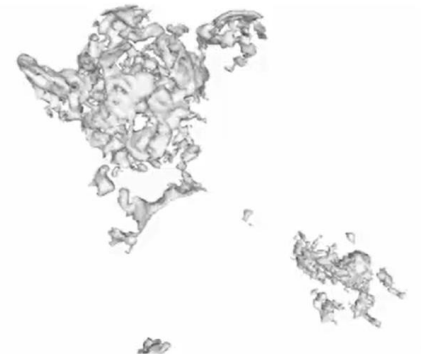
$\sigma = 10$



$\sigma = 50$



$\sigma = 100$



$\sigma = 500$

NeuS

- Neural network to represent an implicit 에스디에프
- To apply a volume rendering, NeuS introduce a probability density function

$$f: \mathbb{R}^3 \rightarrow \mathbb{R}$$

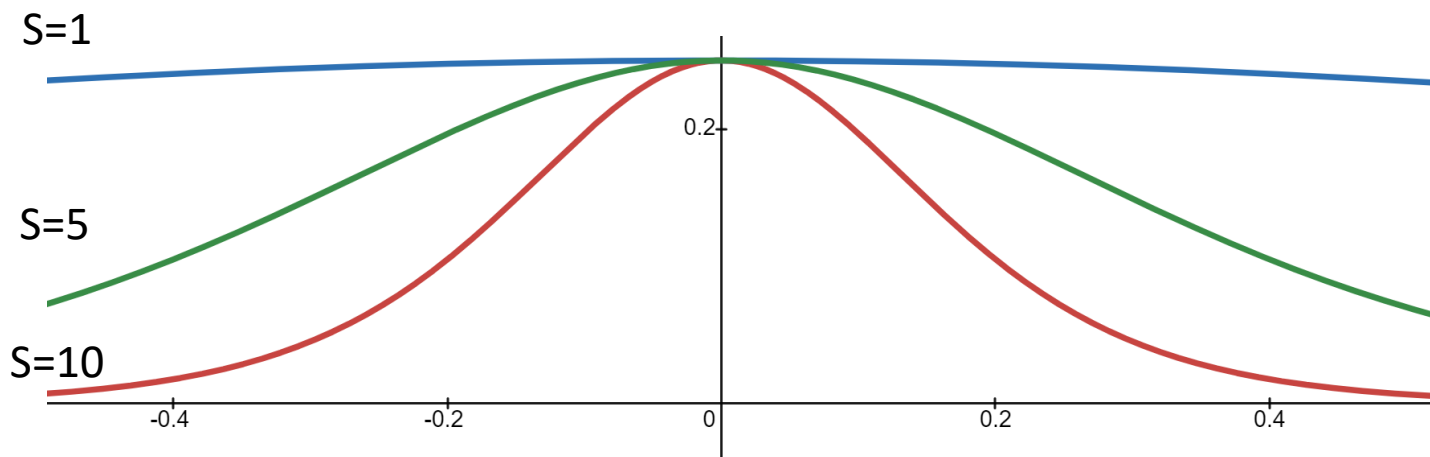
에스디에프 by neural
network

$$\phi_s(f(x))$$

Logistic density distribution
(the derivative of Sigmoid function)

$$\phi_s(x) = \frac{se^{-sx}}{(1 + e^{-sx})^2}$$

$s \rightarrow 0$ when convergence



NeuS

- A novel volumetric rendering

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N \overset{\text{Accumulated Transmittance}}{T_i} \overset{\text{Opacity}}{\alpha_i} c_i$$

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

$$\alpha_i = \max \left(\frac{\Phi_s(f(x_i)) - \Phi_s(f(x_{i+1}))}{\Phi_s(f(x_i))}, 0 \right)$$

$$\Phi'_s(x) = \phi_s(x)$$

NeuS

- Training

$$L = L_{color} + \lambda L_{reg} + \beta L_{mask}$$

$$L_{reg} = \mathbb{E}[(\|\nabla_x f(x)\|_2 - 1)^2]$$

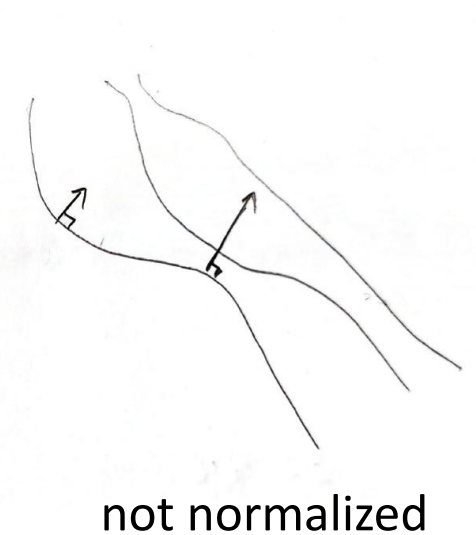
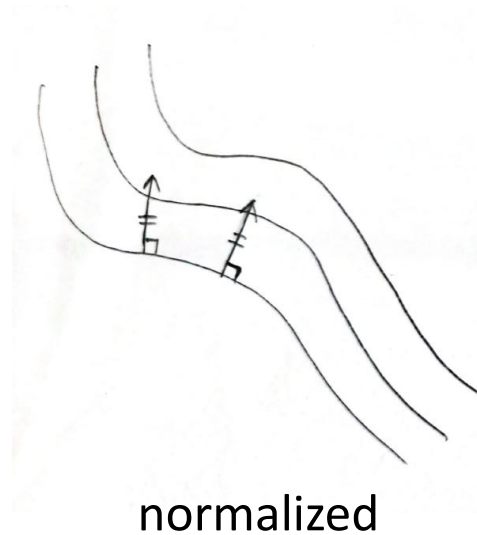
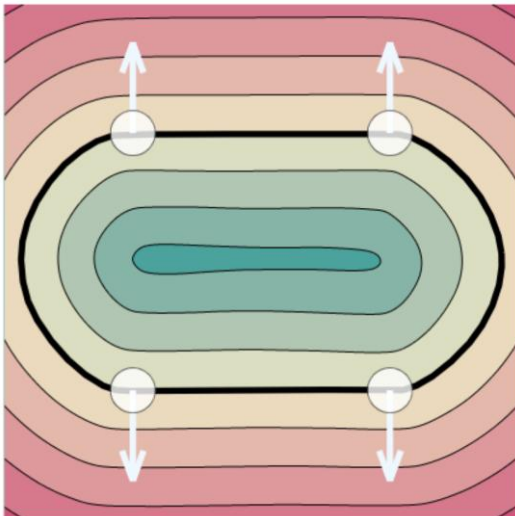
Eikonal Regularizer

$$L_{mask} = BCE$$

Optional Mask

NeuS

- Eikonal regularization normalizes the **magnitude of the normal vectors** everywhere.
- Smooth surface between each level set.



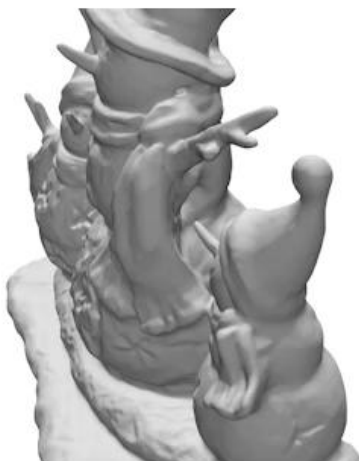
NeuS



Colmap



NeRF



Our geometry
(foreground only)

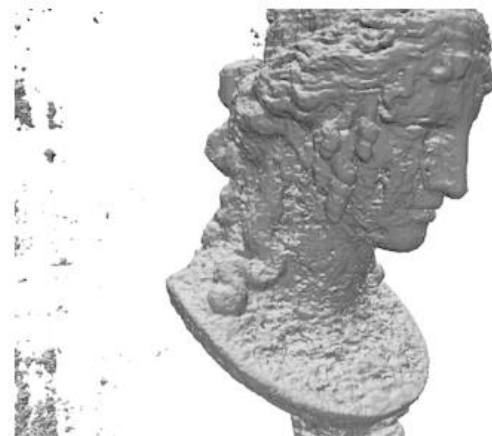


Our rendering
(foreground only)

NeuS



Colmap



NeRF



Our geometry
(foreground only)



Our rendering
(foreground only)

Neuralangelo

- NeuS is slow due to MLP! -> instant-NGP
- Numerical gradients instead of auto-diff for Eikonal regularization
- Curvature loss to further encourage smoothness

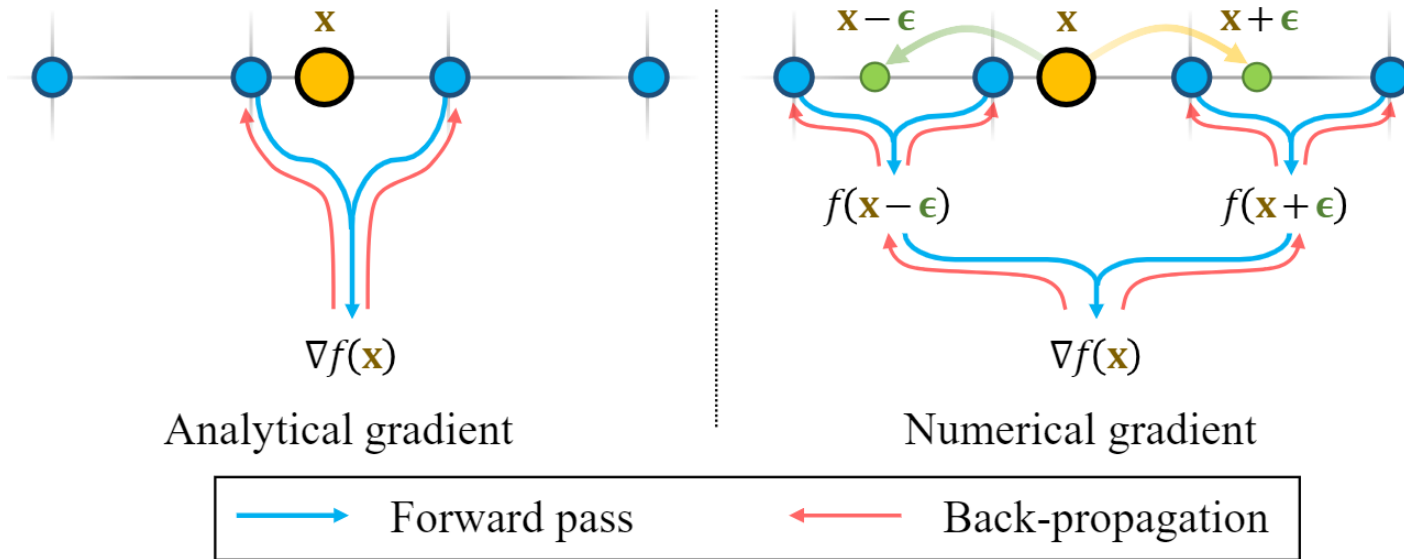
$$L = L_{RGB} + \omega_{eik} L_{eik} + \omega_{curv} L_{curv}$$

$$L_{curv} = \mathbb{E}[|\nabla^2 f(x)|]$$

Neuralangelo

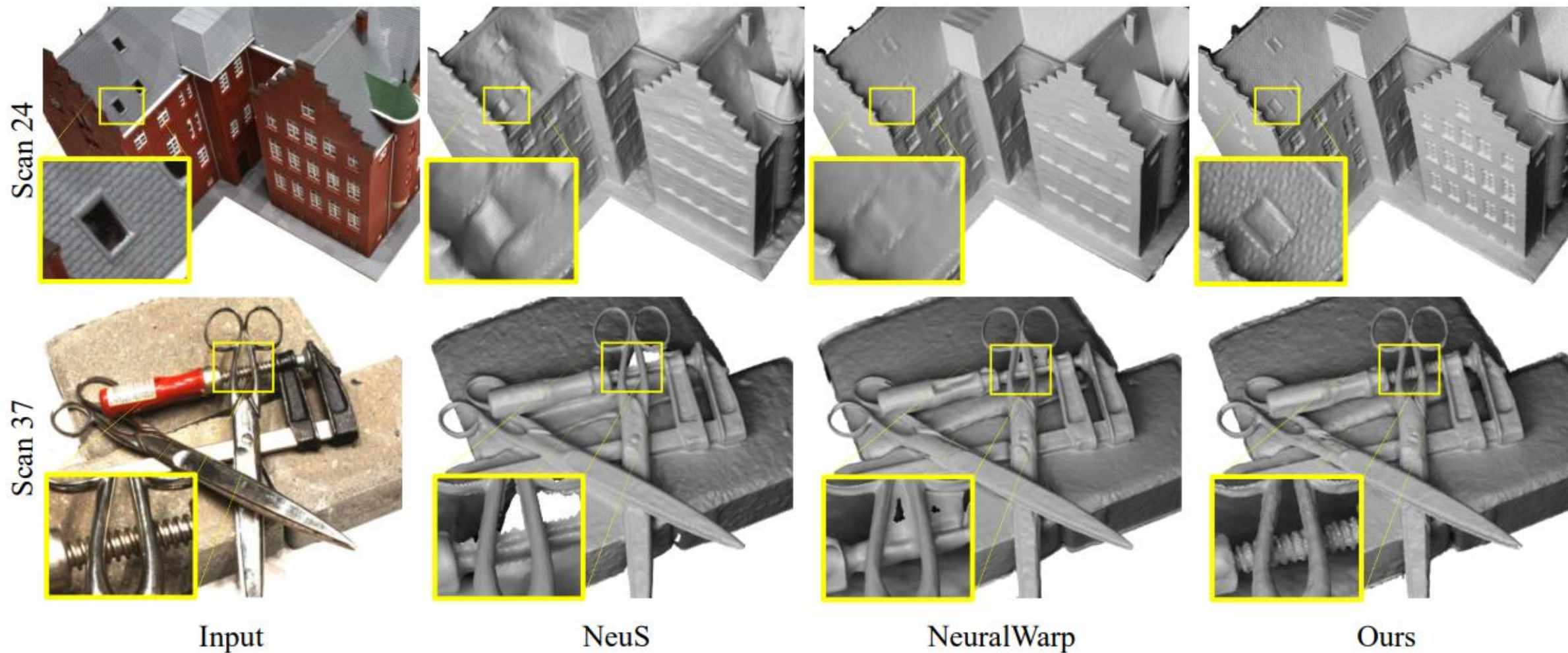
- Numerical gradients

$$\nabla_s f(x_i) = \frac{f(\gamma(x_i + \epsilon)) - f(\gamma(x_i - \epsilon))}{2\epsilon}$$



Start from large ϵ and decrease ϵ
-> **Progressive training**

Neuralangelo



Neuralangelo on DTU Dataset

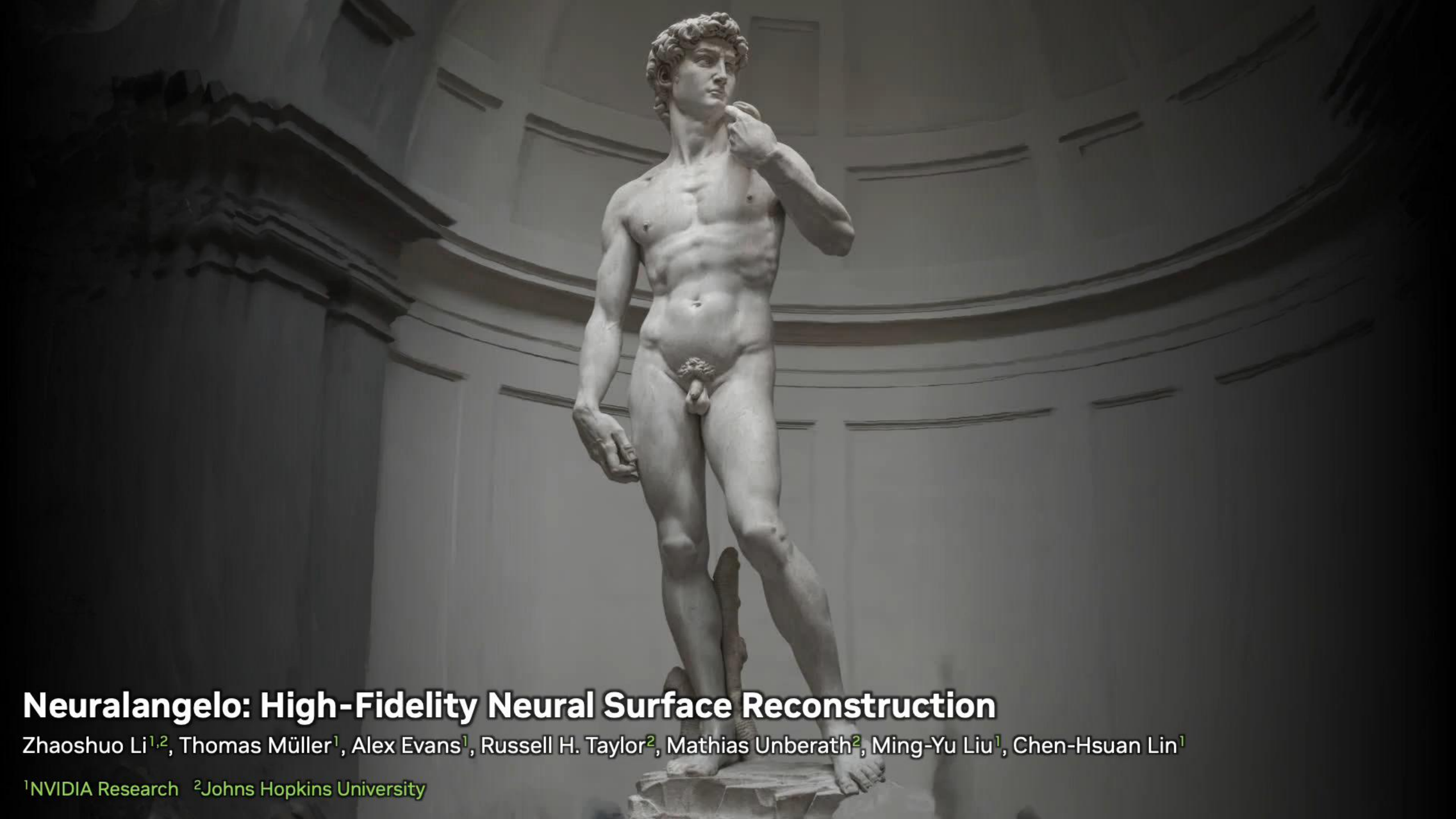
		24	37	40	55	63	65	69	83	97	105	106	110	114	118	122	Mean
Chamfer distance (mm) ↓	NeRF [22]	1.90	1.60	1.85	0.58	2.28	1.27	1.47	1.67	2.05	1.07	0.88	2.53	1.06	1.15	0.96	1.49
	VolSDF [41]	1.14	1.26	0.81	0.49	1.25	0.70	0.72	1.29	1.18	0.70	0.66	1.08	0.42	0.61	0.55	0.86
	NeuS [36]	1.00	1.37	0.93	0.43	1.10	0.65	0.57	1.48	1.09	0.83	0.52	1.20	0.35	0.49	0.54	0.84
	HF-NeuS [38]	0.76	1.32	0.70	0.39	1.06	0.63	0.63	1.15	1.12	0.80	0.52	1.22	0.33	0.49	0.50	0.77
	RegSDF [45] †	0.60	1.41	0.64	0.43	1.34	0.62	0.60	0.90	0.92	1.02	0.60	0.59	0.30	0.41	0.39	0.72
	NeuralWarp [3] †	0.49	0.71	0.38	0.38	0.79	0.81	0.82	1.20	1.06	0.68	0.66	0.74	0.41	0.63	0.51	0.68
	AG	0.67	1.04	0.84	0.39	1.43	1.23	1.11	1.24	1.54	0.85	0.50	1.01	0.37	0.51	0.44	0.88
	AG+P	0.59	0.95	0.46	0.34	1.19	0.70	0.79	1.19	1.37	0.69	0.49	0.93	0.33	0.44	0.44	0.73
	NG	0.48	0.81	0.43	0.35	0.89	0.71	0.61	1.26	1.06	0.74	0.47	0.79	0.33	0.45	0.43	0.65
	NG+P (Ours)	0.37	0.72	0.35	0.35	0.87	0.54	0.53	1.29	0.97	0.73	0.47	0.74	0.32	0.41	0.43	0.61
PSNR ↑	RegSDF [45] †	24.78	23.06	23.47	22.21	28.57	25.53	21.81	28.89	26.81	27.91	24.71	25.13	26.84	21.67	28.25	25.31
	NeuS [36]	26.62	23.64	26.43	25.59	30.61	32.83	29.24	33.71	26.85	31.97	32.18	28.92	28.41	35.00	34.81	29.79
	VolSDF [41]	26.28	25.61	26.55	26.76	31.57	31.50	29.38	33.23	28.03	32.13	33.16	31.49	30.33	34.90	34.75	30.38
	NeRF [22]	26.24	25.74	26.79	27.57	31.96	31.50	29.58	32.78	28.35	32.08	33.49	31.54	31.00	35.59	35.51	30.65
	AG	29.97	24.98	23.11	30.27	30.60	31.27	29.27	34.22	27.47	33.09	33.85	29.98	29.41	35.69	35.11	30.55
	AG+P	30.12	24.63	29.59	30.29	31.60	32.04	29.85	34.19	27.82	33.23	33.95	29.15	29.44	35.99	35.67	31.17
	NG	30.34	25.14	30.20	30.79	31.72	31.86	29.81	34.36	28.01	33.45	34.38	30.39	29.88	36.02	35.74	31.47
	NG+P (Ours)	30.64	27.78	32.70	34.18	35.15	35.89	31.47	36.82	30.13	35.92	36.61	32.60	31.20	38.41	38.05	33.84



Neuralangelo: High-Fidelity Neural Surface Reconstruction

Zhaoshuo Li^{1,2}, Thomas Müller¹, Alex Evans¹, Russell H. Taylor², Mathias Unberath², Ming-Yu Liu¹, Chen-Hsuan Lin¹

¹NVIDIA Research ²Johns Hopkins University



Neuralangelo: High-Fidelity Neural Surface Reconstruction

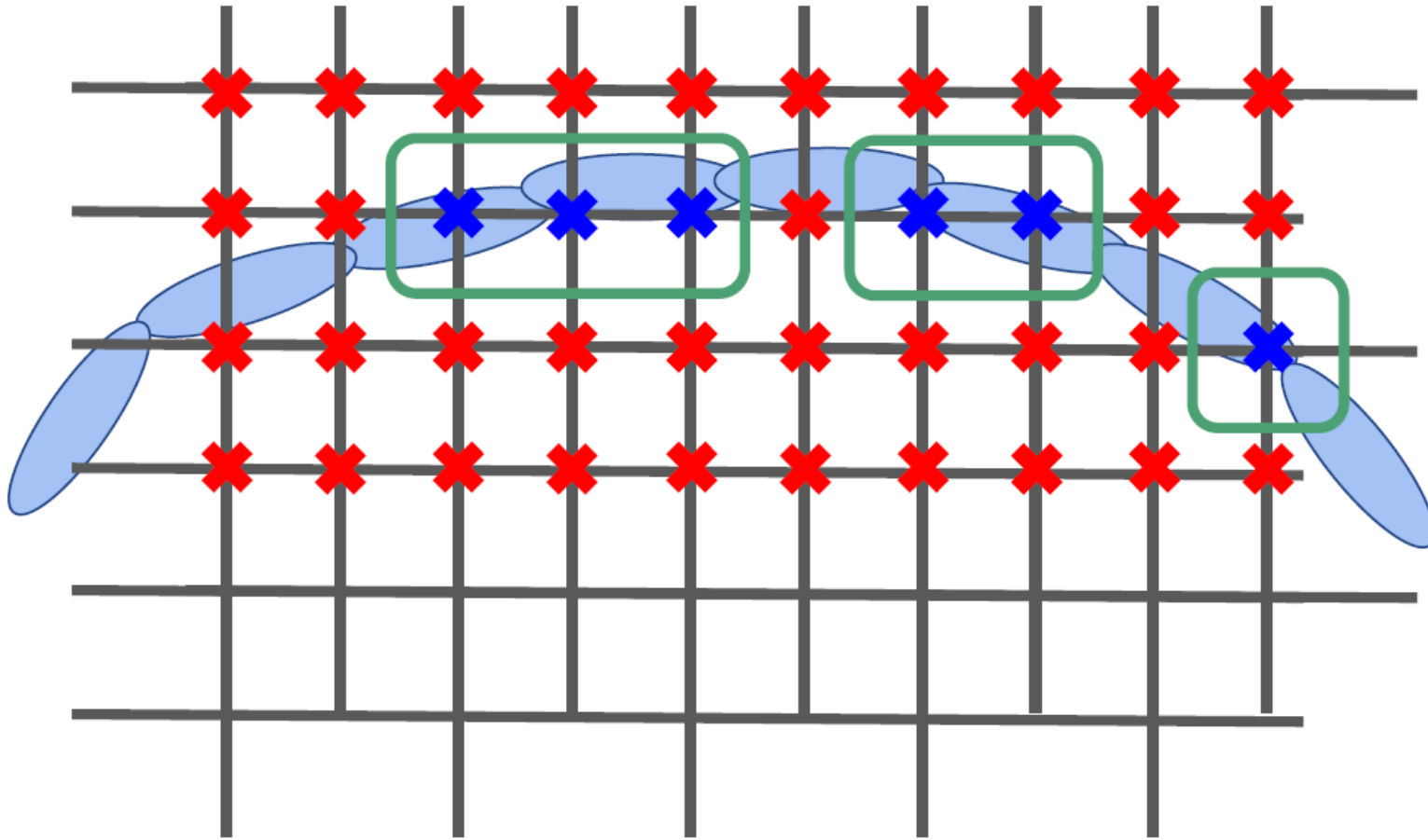
Zhaoshuo Li^{1,2}, Thomas Müller¹, Alex Evans¹, Russell H. Taylor², Mathias Unberath², Ming-Yu Liu¹, Chen-Hsuan Lin¹

¹NVIDIA Research ²Johns Hopkins University

3D Reconstruction using 3DGS

- NeRF-based meshing performs very well, but slow....

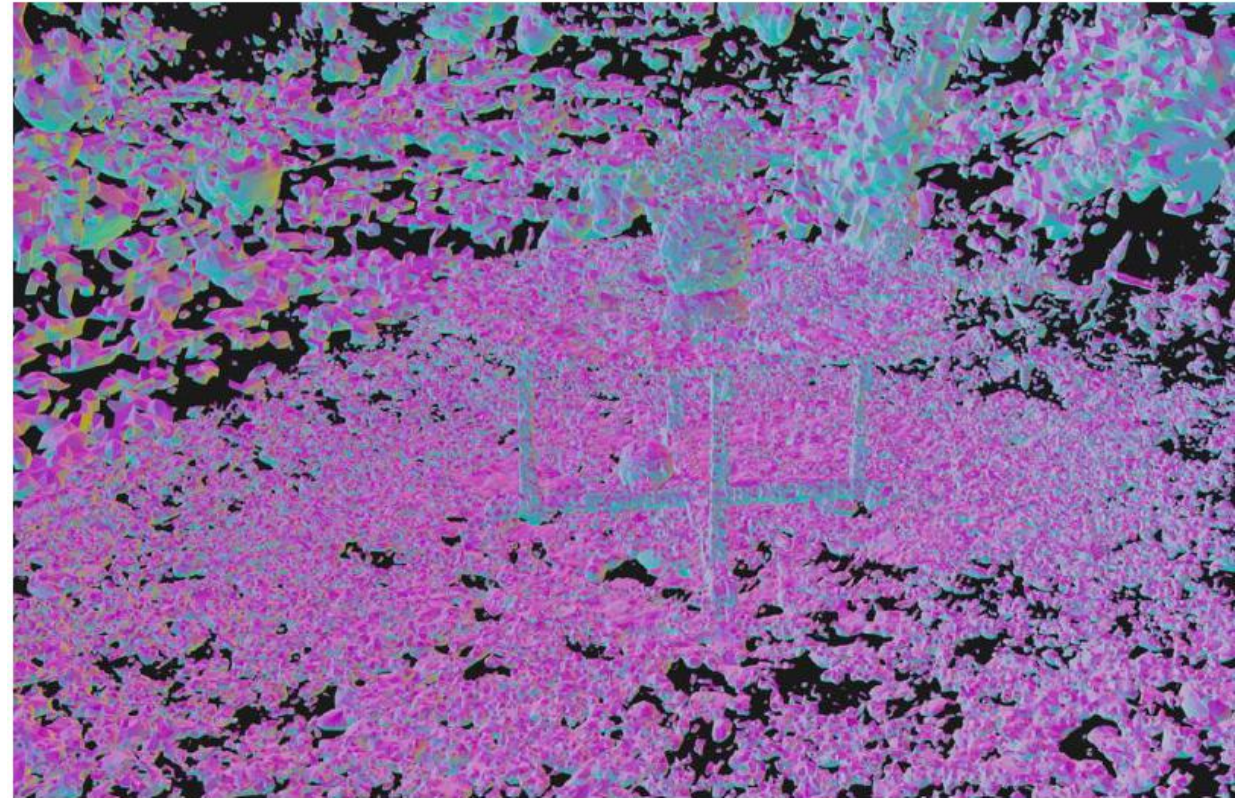
Meshing 3DGS using Marching Cube



$$C = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

$$\alpha_i = o_i e^{-\frac{1}{2}(x - \mu_i)^T \Sigma_{2d} (x - \mu_i)}$$

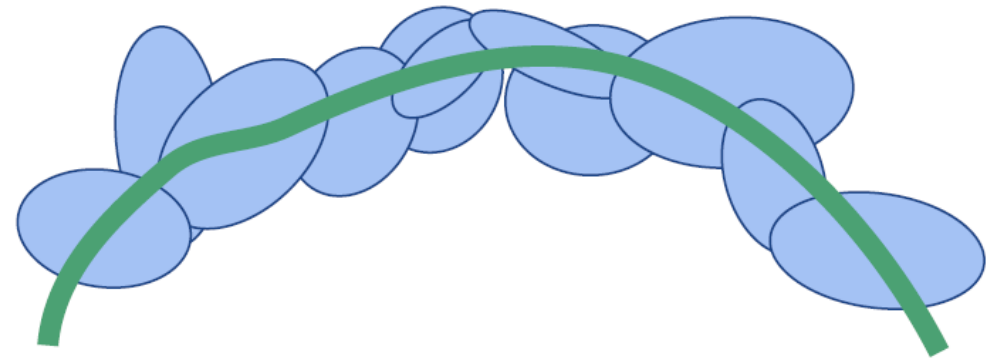
Meshing 3DGS using Marching Cube



SuGaR

- Aligning Gaussians on the surface

$$d(p) = \sum_g \alpha_g \exp \left(-\frac{1}{2} (p - \mu_g)^T \Sigma_g^{-1} (p - \mu_g) \right)$$

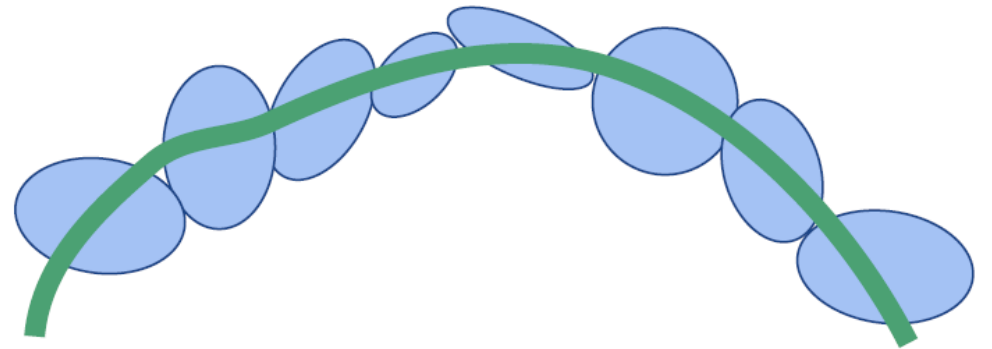


SuGaR

- Aligning Gaussians on the surface

#1: Gaussians should have limited overlap, and be well-spread on the surface.

$$d(p) = \alpha_{g^*} \exp \left(-\frac{1}{2} (p - \mu_{g^*})^T \Sigma_{g^*}^{-1} (p - \mu_{g^*}) \right)$$

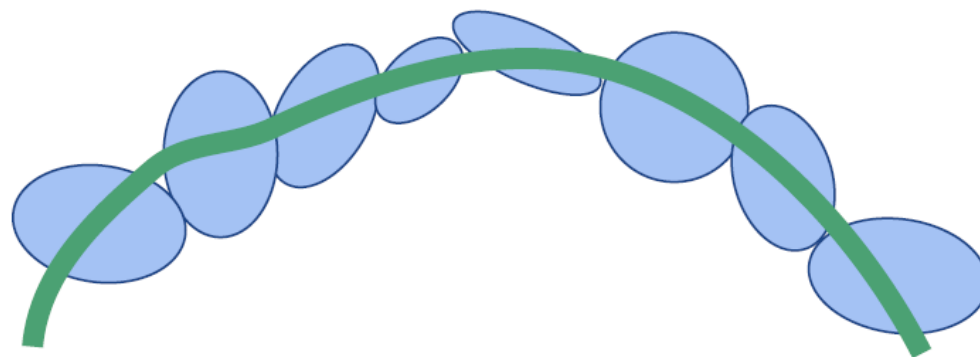


SuGaR

- Aligning Gaussians on the surface

#2: Gaussians should be opaque or fully transparent (otherwise, isosurface are meaningless).

$$d(p) = \exp \left(-\frac{1}{2} (p - \mu_{g^*})^T \Sigma_{g^*}^{-1} (p - \mu_{g^*}) \right)$$



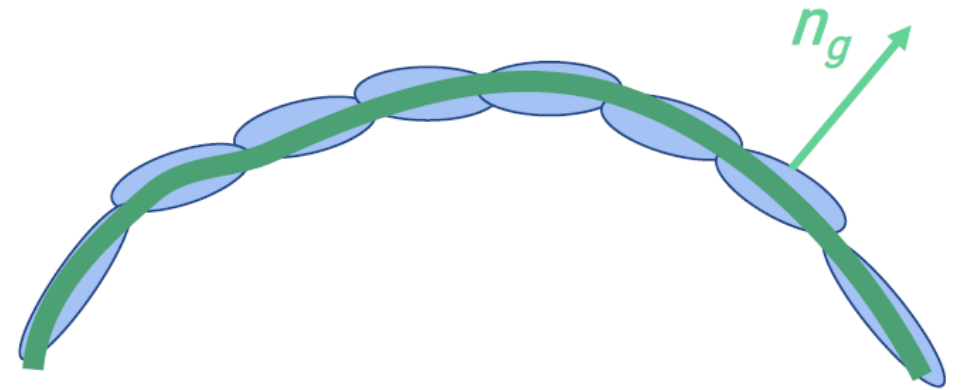
SuGaR

- Aligning Gaussians on the surface

#3: Gaussians should be as flat as possible (one of the three scaling factors should be close to zero).

$$\bar{d}(p) = \exp \left(-\frac{1}{2s_{g^*}^2} \langle p - \mu_{g^*}, n_{g^*} \rangle^2 \right)$$

because $(p - \mu_g)^T \Sigma_g^{-1} (p - \mu_g) \approx \frac{1}{s_g^2} \langle p - \mu_g, n_g \rangle^2$



SuGaR

- Aligning Gaussians on the surface

Original density

$$d(p) = \sum_g \alpha_g \exp \left(-\frac{1}{2} (p - \mu_g)^T \Sigma_g^{-1} (p - \mu_g) \right)$$

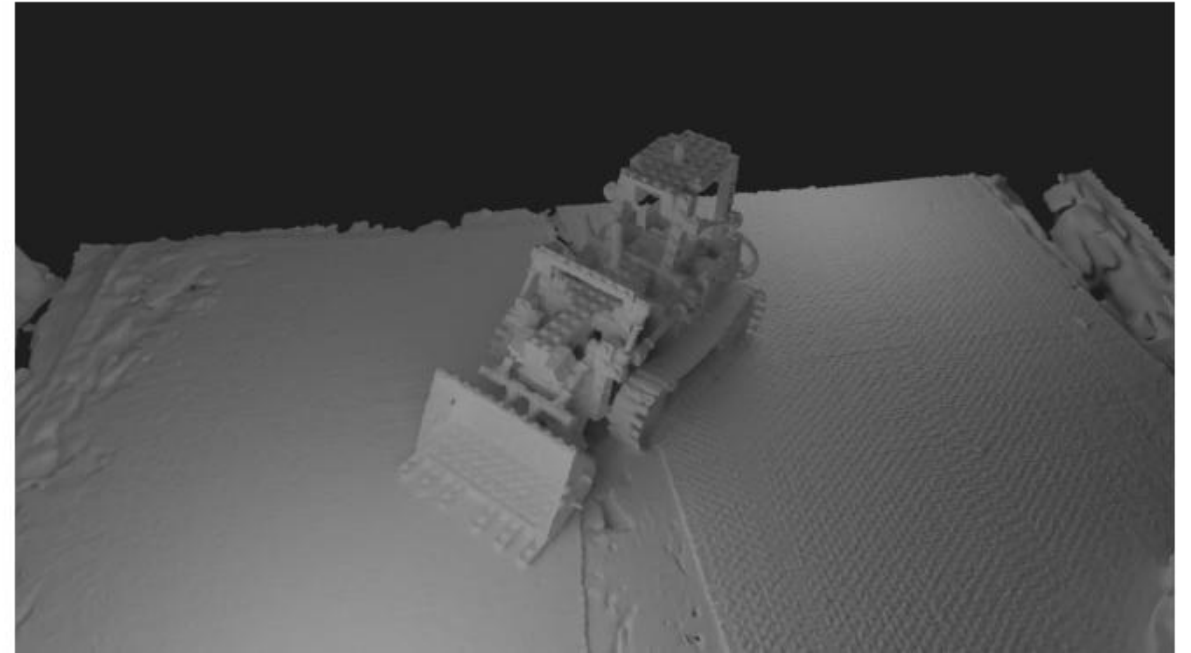
$$\mathcal{R} = |d(p) - \bar{d}(p)|$$

Desired density

$$\bar{d}(p) = \exp \left(-\frac{1}{2s_{g^*}^2} \langle p - \mu_{g^*}, n_{g^*} \rangle^2 \right)$$

SuGaR

- Aligning Gaussians on the surface + Poisson reconstruction + Mesh refinement



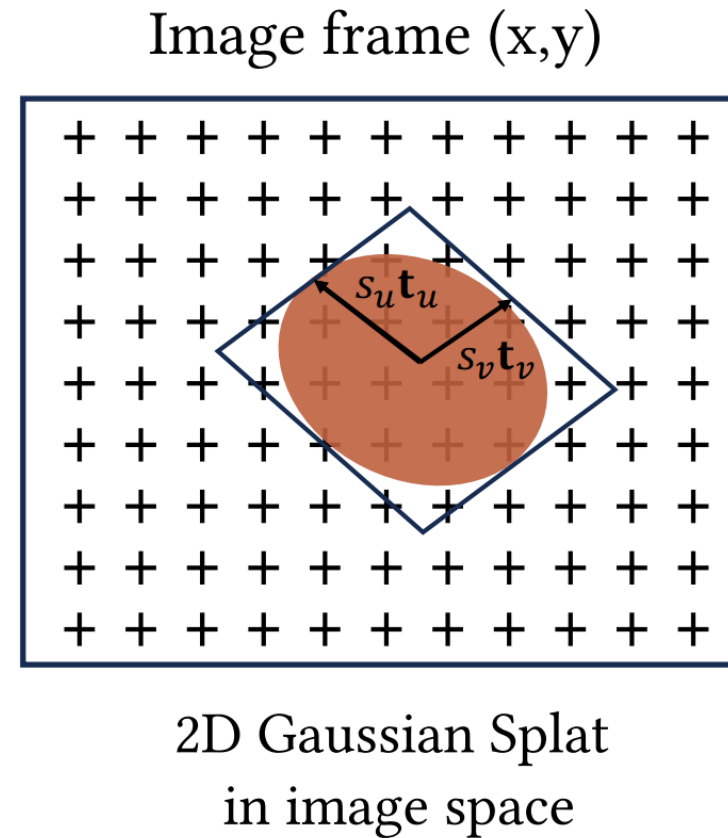
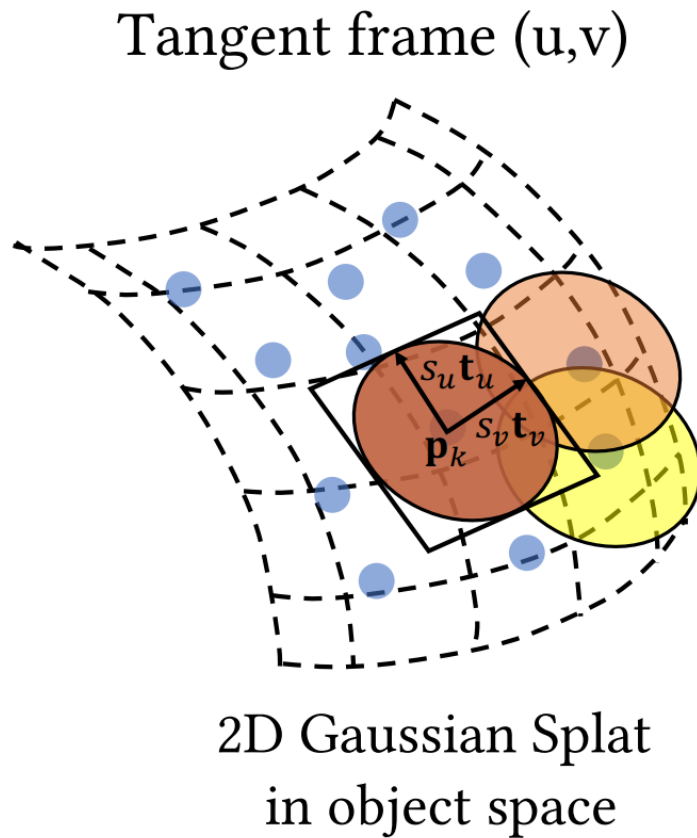
SuGaR

- Aligning Gaussians on the surface + Poisson reconstruction + Mesh refinement



2DGS

- 2D elliptical disks as scene representation



2DGS

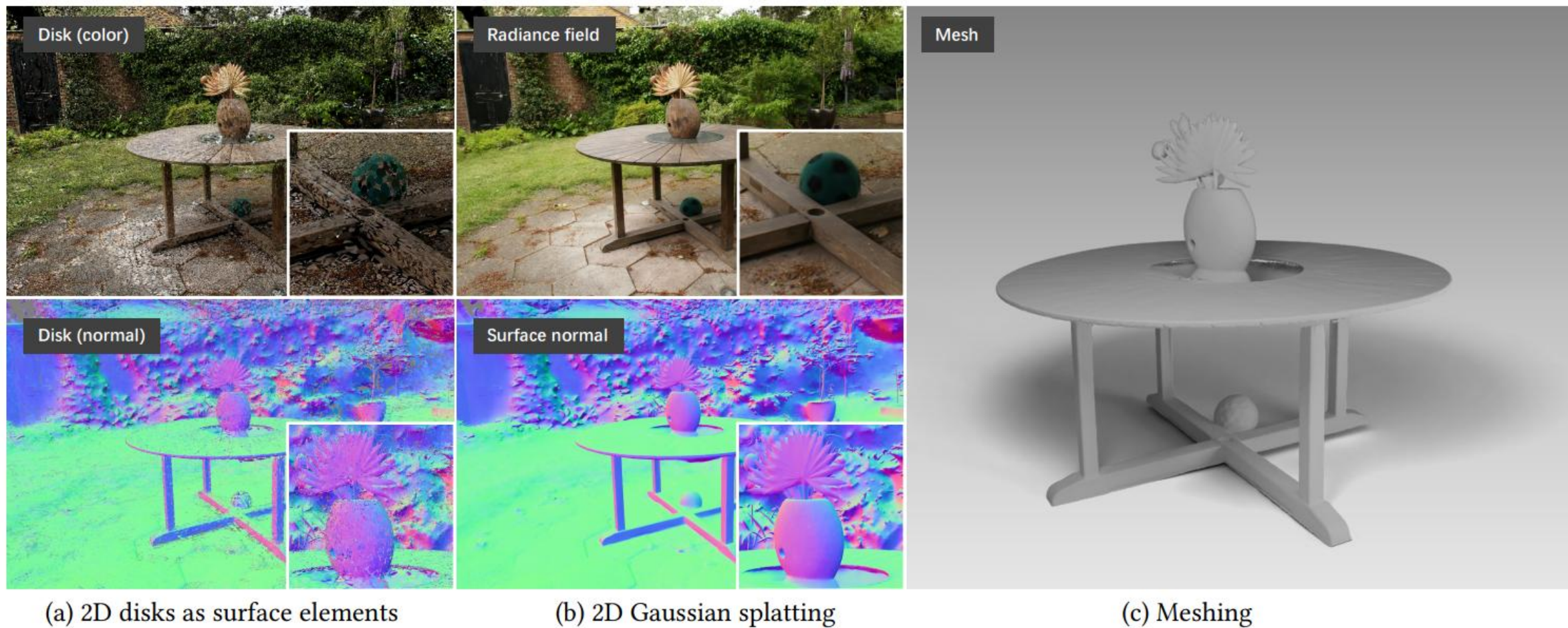
- Training with depth distortion and normal consistency

$$L = L_{RGB} + \alpha L_d + \beta L_n$$

$$L_d = \sum_{i,j} \omega_i \omega_j |z_i - z_j|$$

$$L_n = \sum_i \omega_i (1 - n_i^\top N)$$

2DGS



결론

본 강의 자료는 연세대학교 박은병교수님의 방송미디어공학회
실감미디어워크숍(2025.04.24.)에서의 발표자료를 정리한 것이며, 본 교과목을 위해 허가를 받고
게시된 것이므로 수업목적 외 용도로 사용할 수 없으며, 무단으로 복제, 배포, 전송 또는 판매하는
행위를 금합니다. 이를 위반 시 민형사상 법적 책임은 행위자 본인에게 있습니다.